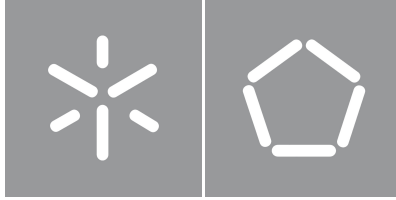


Universidade do Minho
Escola de Engenharia

Embedded Systems

Compiler

**Linguagens de
Sistemas Embebidos**



Universidade do Minho
Escola de Engenharia

Embedded Systems

Compiler

**Linguagens de
Sistemas Embebidos**

Master's Project
Master's Degree in Industrial Electronics and Computers Engineering

Oriented by
Prof. Adriano Tavares

Contents

I	Introduction Material	1
1	Introduction	2
1.1	State of Art	2
1.2	Motivation and Relevance	3
1.3	Project Goals	3
2	Analysis	5
2.1	Problem Statement	5
2.2	Requirements and Constraints	6
2.2.1	Requirements	6
2.2.2	Constraints	7
2.3	System Overview	7
3	Design	9
3.1	Lexical Analysis(Lexer)	9
3.1.1	Design Objectives	9
3.1.2	Input and Output Requirements	10
3.1.3	Modular Architecture and Design Criteria	10
3.1.4	Design Strategies	11
3.1.5	Summary	11
3.2	Syntactic Analysis(Parser)	11
3.2.1	Parser - Grammar Rules	11
3.2.2	AST(Abstract Syntax Tree)	14
3.3	Semantic Analysis	24
3.3.1	Symbol Table	24

3.3.2	Type Checking	31
3.4	Code Generation	34
3.4.1	Architecture and Role of the Back-End	34
3.4.2	Symbol Table Integration	35
3.4.3	Recursive Tree Traversal and Node Categorization	35
3.4.4	Memory Allocation and Variable Management	36
3.4.5	Stack-Based Parameter Passing and Calling Conventions	37
3.4.6	Register Usage for Temporary Data and Type Handling	37
3.4.7	Translation of Control Structures	37
3.4.8	Memory management	38
3.4.9	Function Code Generation and Return Semantics	38
4	Implementation	39
4.1	Lexical Analysis(Lexer)	39
4.1.1	Implementation of Scanner Rules	39
4.2	Syntactic Analysis(Parser)	45
4.2.1	Parser - Grammar Rules	45
4.2.2	AST(Abstract Syntax Tree)	53
4.3	Semantic Analysis	67
4.3.1	Symbol Table	67
4.3.2	Type Checking	77
4.4	Code Generation	97
4.4.1	Function codegen	98
4.4.2	Function linegen	99
4.4.3	Function genFunc	101
4.4.4	Function genStmt	105
4.4.5	Function genExp	107
5	Tests and Validation	111
5.1	AST Test	111
5.2	Semantic Test	116
5.3	Dry Run	120
6	Conclusion and Future work	139

List of Figures

1	Compiler System Overview	8
2	CreateNode() Flowchart	17
3	Addchild() Flowchart	18
4	AddSiblings() Flowchart	18
5	Function Definition AST	19
6	For Loop AST	19
7	If Statement AST	20
8	Variable Initialization AST	20
9	Const Variable AST	21
10	Function Call AST	21
11	Simple representation of tree node pointers to each node's scope table.	26
12	Symbol tables created for the example, their scopes and relationship to each other. . . .	27
13	Flowchart for the createTable function.	28
14	Flowchart for the st_insert function.	29
15	Flowchart for the st_lookup function.	30
16	Flowchart for the st_lookupLocal function.	30
17	Memory management for code generation	38
18	Function Tree	60
19	For Tree	61
20	If Tree	62
21	Variable Initialization Tree	64
22	Const Variable Tree	65
23	Function Call Tree	66
24	Flowchart for the checkNode function.	78

25	Generated Tree	113
26	Generated Tree	113
27	Generated Tree	115
28	Generated Tree	115
29	Compiler output showing detection of redeclaration of a variable within the same scope.	116
30	Compiler output showing detection of the use of an undeclared variable.	117
31	Compiler output showing correct management of shadowed variables in nested scopes.	118
32	Compiler output showing detection of a mismatch between function declaration and definition.	119
33	Compiler output showing detection of the use of <code>break</code> outside a loop or switch.	120

List of Tables

Acronyms

AST Abstract Syntax Tree.

ISA Instruction Set Architecture.

LLVM Low Level Virtual Machine.

Part I

Introduction Material

Chapter 1

Introduction

Compilers are fundamental tools in software development, enabling the transformation of source code written in high-level languages into machine-executable instructions. Over the years, compiler design has evolved into a mature field, with established methodologies and toolchains.

This project focuses on building a compiler front-end for a simplified version of the C programming language. It aims to explore the key concepts behind compiler construction, including lexical, syntactic, and semantic analysis. Additionally, the compiler is designed to target the 8051 Instruction Set Architecture (**ISA**), a widely used architecture in embedded systems, making the project not only theoretically relevant but also practically applicable in real-world constrained environments.

The following sections present a contextual overview of existing technologies, the motivation behind the project, its goals, and relevant terminology.

1.1 State of Art

Compiler construction has been extensively studied and refined over the last decades, resulting in a well-established field supported by robust theoretical foundations and practical tools. Traditional compiler architectures are typically divided into several stages: lexical analysis, syntax analysis, semantic analysis, optimization, and code generation.

In educational and prototyping contexts, tools such as **Flex** (Fast Lexical Analyzer Generator) and **Bison** (a parser generator compatible with Yacc) are widely used to automate the development of the lexical and syntactic phases. Flex allows the specification of regular expressions for token recognition, while Bison provides a declarative way to define context-free grammars and generate the corresponding Abstract Syntax Tree (AST).

Many simplified or pedagogical compilers follow this model to demonstrate the principles of language translation. These include academic projects that replicate parts of the C language or other imperative

languages. Additionally, open-source compilers such as **TinyCC** and subsets of **Clang/LLVM** are often studied for their modularity and extensibility.

This project aligns with these foundations by implementing the front end of a C-like compiler using Flex and Bison. The emphasis is placed on supporting fundamental language features such as declarations, expressions, control structures, and function calls while constructing a corresponding abstract syntax tree for further analysis or transformation.

1.2 Motivation and Relevance

The development of a compiler is a fundamental exercise in understanding how programming languages work beneath the surface. It involves the application of multiple computer science concepts such as formal languages, data structures, parsing algorithms, and code generation strategies.

This project is motivated by the desire to gain a deeper, practical understanding of how a high-level language like C is transformed into an internal representation that can be executed or analyzed. By building a compiler front-end using Flex and Bison, the project explores the key mechanisms behind lexical and syntactic analysis, as well as the construction of an Abstract Syntax Tree (**AST**) — a critical structure used in semantic analysis, optimization, and code generation.

The relevance of this work lies not only in the technical knowledge it consolidates but also in its potential use as an educational tool. The compiler implements a simplified subset of the C language, making it accessible for further expansion or adaptation in future projects. It also provides a practical foundation for integrating more advanced compiler components, such as semantic checks or intermediate code generation.

In addition, understanding compiler design is essential in domains such as language design, embedded systems, and static analysis tools.

1.3 Project Goals

The main goal of this project is to design and implement a compiler front-end for a simplified subset of the C programming language. The compiler focuses on the lexical, syntactic, and basic semantic stages of compilation, with an emphasis on producing a well-structured Abstract Syntax Tree (AST).

To achieve this, the following specific objectives were defined:

- **Design a custom grammar** that accurately describes a relevant subset of the C language, in-

cluding variable declarations, expressions, control structures, and functions.

- **Implement a lexical analyzer** using Flex that can correctly tokenize source code and handle C-specific tokens, such as keywords, identifiers, literals, operators, and comments.
- **Develop a syntax analyzer** using Bison based on the designed grammar, capable of detecting syntax errors and constructing the corresponding AST.
- **Define and build the AST** structure in C, ensuring that it reflects the syntactic structure of the input and supports traversal, debugging, and further analysis.
- **Support for scope-aware declarations and initialization**, including handling of modifiers like `const`, `static`, and pointer declarations.
- **Document and validate the implementation** through extensive testing of valid and invalid code examples, showcasing the compiler's capabilities.

These goals ensure a modular and extensible architecture, preparing the foundation for future enhancements such as semantic analysis, type checking, or even code generation.

Chapter 2

Analysis

This chapter presents a detailed breakdown of the main components of the compiler developed in this project. The analysis is structured around the traditional compilation stages: lexical, syntactic, semantic, and code generation. Each section focuses on how the respective phase was implemented, highlighting the tools, data structures, and strategies used to process the source code from raw text to an intermediate representation. The goal is to provide a clear understanding of the internal mechanics of the compiler and how each stage contributes to the transformation and validation of the input program.

2.1 Problem Statement

Despite the widespread use of high-level programming languages such as C, the process by which human-readable code is translated into low-level instructions for specific hardware platforms remains complex and often abstract to many developers. In embedded systems, where resource constraints and hardware-specific instructions are critical, understanding and controlling this transformation becomes even more important.

This project addresses the need for a custom compiler front-end capable of translating a simplified subset of the C programming language into an intermediate representation compatible with the 8051 Instruction Set Architecture (ISA). The challenge lies in designing a modular and extensible compilation pipeline that supports lexical, syntactic, and semantic analysis while being tailored to the characteristics of the 8051 microcontroller family.

The lack of educational tools that bridge the gap between high-level language design and low-level hardware-specific code motivates the creation of a minimal yet functional compiler that can serve both learning and prototyping purposes. This includes implementing core features such as variable declarations, expressions, control structures, function calls, and a robust Abstract Syntax Tree (AST) to facilitate further analysis or code generation.

2.2 Requirements and Constraints

This section outlines the key requirements and constraints that shaped the development of the compiler. It distinguishes between functional and non-functional requirements, as well as technical and non-technical constraints.

These considerations ensure the system meets its intended purpose—parsing and analyzing a subset of C for the 8051 ISA—while remaining modular, extensible, and suitable for academic use.

2.2.1 Requirements

The requirements define what the compiler must achieve to be considered functionally complete. They include the expected behavior of each compiler phase and outline performance, usability, and maintainability expectations.

Functional

- Analyze the source code.
- Generate the Abstract Syntax Tree (AST).
- Detect lexical and syntactic errors.
- Support declarations, expressions, and control structures.
- Produce code targeting the 8051 ISA.

Non-Functional

- Ensure modular and maintainable design.
- Maintain readable and well-structured code.
- Allow future expansion for semantic analysis or optimization.
- Guarantee reasonable compilation times.

2.2.2 Constraints

This section outlines the limitations and conditions that influence the development and scope of the compiler project. These constraints must be considered throughout the design and implementation phases to ensure feasibility and alignment with the project goals.

Technical

- Target architecture is limited to the 8051 Instruction Set Architecture (ISA).
- Only a subset of the C language is supported.
- The compiler front-end is developed using Flex and Bison.
- The Abstract Syntax Tree must be implemented in C.
- No external code generation frameworks or libraries are allowed.

Non-Technical

- The project must be completed within the academic semester timeline.
- Groups of four elements with an assigned leader.
- The project should rotate within all groups in the class.
- The compiler should be accessible for evaluation and testing by instructors.

2.3 System Overview

This section presents a high-level view of the compiler architecture. The system is divided into modular components corresponding to the main stages of compilation: lexical analysis, syntactic analysis, semantic analysis, and code generation. Each stage processes the input incrementally, transforming it from source code into executable instructions targeting the 8051 architecture.

The following diagram illustrates the overall structure and data flow between components.

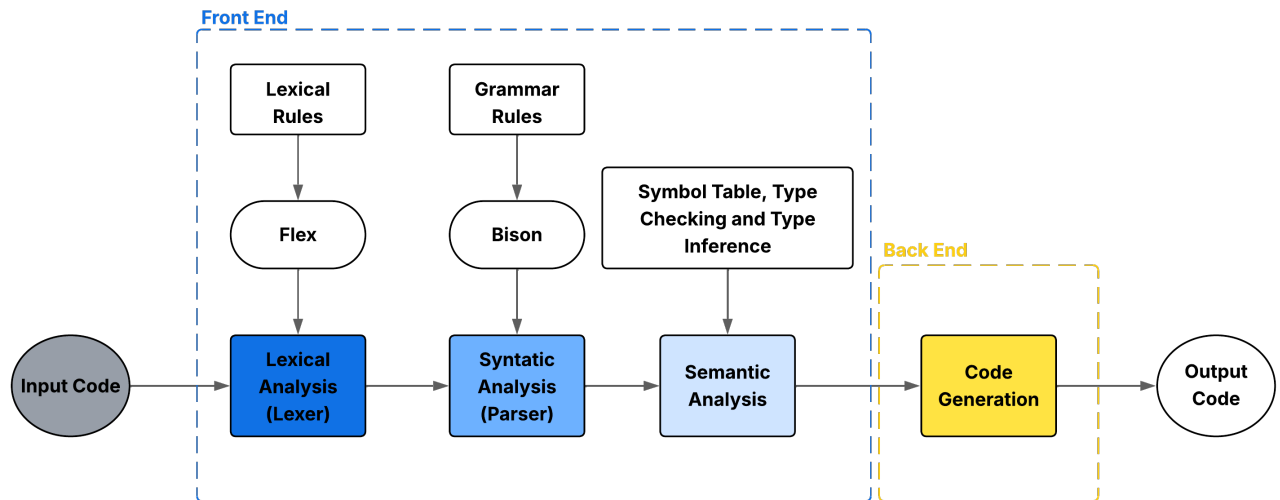


Figure 1: Compiler System Overview

Figure 1 illustrates the high-level architecture of the compiler. The compilation process is divided into two major components: the Front End and the Back End.

In the Front End, the source code is first processed by the Lexical Analyzer (implemented using Flex), which identifies tokens based on lexical rules. These tokens are then passed to the Syntactic Analyzer (powered by Bison), where the grammar rules guide the construction of the Abstract Syntax Tree (AST). After parsing, the Semantic Analysis stage checks for correctness using a symbol table and performs type checking and inference.

The Back End handles Code Generation, transforming the semantically validated AST into target code compatible with the 8051 Instruction Set Architecture (ISA). The final result is a low-level output ready for execution on 8051-based microcontrollers.

Chapter 3

Design

This chapter presents the architectural and structural decisions made during the development of the compiler. It describes how the system components interact, how the data flows through the compilation pipeline, and how each stage is implemented to meet the project's requirements.

The design follows a modular approach, separating concerns between lexical, syntactic, semantic, and code generation phases. Each module is responsible for a specific layer of abstraction and contributes to the transformation of the input source code into executable output targeting the 8051 instruction set architecture.

3.1 Lexical Analysis(Lexer)

The design phase of the lexical analyzer aims to transform the lexical analysis requirements for the C language (with 8051 extensions) into a structured set of specifications and architectural choices, which will guide the development and ensure a reliable, modular, and extensible system.

3.1.1 Design Objectives

The lexical analyzer for the C compiler targeting the 8051 should:

- Read the source code as a sequence of characters;
- Identify and separate valid tokens according to the lexical grammar of the language (keywords, identifiers, literals, operators, symbols, etc.);
- Detect and signal invalid patterns, ensuring rigorous code validation from the earliest phases;
- Ignore whitespace, comments, and other elements irrelevant to the syntax;
- Be modular and easily extensible for new keywords, operators, or language constructs.

3.1.2 Input and Output Requirements

Input:

- Sequence of characters from the source code to be analyzed.

Output:

- Sequence of tokens, each identified by its type (keyword, identifier, number, operator, literal, etc.) and associated value when applicable (e.g., variable names, numeric values, strings).
- Signaling of detected lexical errors (e.g., invalid identifiers, unterminated strings).

3.1.3 Modular Architecture and Design Criteria

The lexical analyzer was developed with a modular architecture, organized into logical blocks of rules, following classic compiler engineering criteria:

- **Token Definition Block:** Regular expressions that identify each type of token (keywords, operators, literals, identifiers, etc.).
- **Actions Block:** Actions associated with each pattern, where the token to be returned and its semantic value are defined.
- **Comment and Whitespace Handling:** Specific rules to ignore comments (single and multi-line), spaces, and line breaks, ensuring that only relevant information is processed.
- **Lexical Error Handling:** Explicitly detects and reports invalid patterns, ensuring errors are signaled early in the process.
- **Integration Interface:** Tokens and their semantic values are provided to the parser in a normalized way, ensuring efficient communication between modules.

The design also adheres to the following fundamental criteria:

- **Lexical Correctness:** Strict alignment with the target language grammar, rejecting invalid sequences;
- **Modularity and Extensibility:** Rules organized into blocks, facilitating maintenance and the future introduction of new lexical patterns;
- **Compatibility:** Efficient communication with the syntactic analyzer and other compiler modules.

3.1.4 Design Strategies

During the scanner's design phase, the following main strategies were defined:

- Use of an automatic lexical analyzer generator (`flex`), allowing for the declarative and efficient definition of complex patterns;
- Structuring of regular expressions by thematic groups (keywords, identifiers, literals, operators, etc.), promoting isolation and clarity of rules;
- Assignment of simple, direct, and efficient actions to each recognized pattern, associating semantic values when necessary;
- Implementation of explicit mechanisms for detection and signaling of lexical errors, facilitating problem diagnosis;
- Clear definition of the interfaces between the scanner and the parser, with structured passing of tokens and respective values.

3.1.5 Summary

This design phase clearly and systematically establishes the objectives, requirements, criteria, and architectural options of the lexical analyzer, defining the path for a modular and easily evolvable implementation, which is fundamental for the overall quality of the compiler.

3.2 Syntactic Analysis(Parser)

The design phase aims to transform the syntactic analyzer requirements into a structured set of specifications, decisions, and plans that will guide development and ensure that the system meets the defined objectives.

3.2.1 Parser - Grammar Rules

This section presents the design of the grammar rules used to define the syntax of the supported C-like language. The grammar was carefully constructed using Bison to ensure it accurately reflects the structure and semantics of C constructs, while remaining simplified for the scope of this project. Each rule defines how sequences of tokens are grouped into higher-level syntactic elements, forming the backbone of the

parser. The rules are also closely integrated with the construction of the Abstract Syntax Tree (AST), enabling the parser to generate a structured representation of the input program as it recognizes valid constructs.

Design Objectives

The syntactic analyzer for the C compiler targeting the 8051 should:

- Receive as input a sequence of tokens produced by the lexical analyzer;
- Identify and structure the syntactic constructs of the C language according to the defined grammar;
- Ensure robustness in the presence of errors, providing informative feedback to the user;
- Prepare the necessary information for the semantic analysis and code generation phases;
- Be modular, extensible, and easily adaptable to future needs.

Input and Output Requirements

Input:

- Sequence of tokens, each identified by its type (keyword, identifier, operator, etc.) and, when applicable, associated value (e.g., variable names, literal values).

Output:

- Hierarchical syntactic structure (for example, Abstract Syntax Tree – AST) representing the source program, and a list of syntactic errors found during analysis.

Modular Architecture and Design Criteria

The syntactic analyzer was developed following a modular architecture, where each component has well-defined responsibilities aligned with the main compiler engineering criteria. This approach aims to facilitate maintenance, enable future extensions, and ensure robust syntactic analysis in accordance with C and 8051 extensions.

The main system blocks are:

- **Token Reception:** Directly interacts with the lexical analyzer, receiving the sequence of tokens to be processed;

- **Parsing:** Processes tokens according to the language grammar, identifying valid syntactic structures and building the abstract syntax tree (AST);
- **Error Handling:** Detects, logs, and attempts to recover from syntactic errors, allowing the reporting of multiple errors per execution;
- **Output and Integration:** Provides the syntactic structure and detected errors to the following compiler phases, ensuring a standardized interface between modules.

The design also follows these fundamental criteria:

- **Correctness:** Full compliance with the target grammar, rejecting invalid constructs;
- **Modularity and Extensibility:** Independent components easily adaptable to new requirements or language constructs;
- **Compatibility:** Efficient and standardized communication with the lexical analyzer and other compiler modules.

Design Strategies

During this phase, the following main actions were defined:

- Use of an automatic parsing approach (Bison), allowing a clear definition of the grammar and facilitating maintenance;
- Clear definition of interfaces between modules (tokens in structured format, output as abstract syntax tree);
- Structuring of the grammar and parser code by thematic blocks (expressions, statements, functions, etc.), favoring isolation of responsibilities;
- Planning of mechanisms for detection and recovery from syntactic errors, to increase system resilience;
- Definition of documentation and usage examples to support validation and maintenance of the parser.

Summary

Thus, the design phase clearly and systematically defines the requirements, criteria, and architectural options of the syntactic analyzer, establishing the paths and principles that will guide all subsequent implementation.

3.2.2 AST(Abstract Syntax Tree)

The design phase aims to transform the syntactic and semantic requirements into a structured representation of the source code. The Abstract Syntax Tree (AST) plays a central role in this process, serving as a hierarchical, tree-based structure that captures the essential components of a program in a language-agnostic way.

This section outlines the design principles behind the AST, detailing how various programming constructs are modeled and connected to support further analysis and transformation throughout the compilation pipeline.

AST Structure Overview

The Abstract Syntax Tree (AST) is designed as a hierarchical structure where each node represents a syntactic construct of the input program. All nodes inherit from a common `TreeNode` structure that contains type information, node kind (e.g., declaration, expression, statement), and links to children and siblings to represent nesting and sequencing.

TreeNode Structure

The `TreeNode` structure represents the fundamental building block of the Abstract Syntax Tree (AST). Each node contains the following fields:

- **pchild:** Pointer to the first child of the node. This represents hierarchical relationships, such as a function containing parameters or a code block containing multiple statements.
- **psibling:** Pointer to the next sibling node at the same level. This allows the representation of sequential elements, such as multiple declarations within a block.
- **lineno:** Stores the line number in the source code from which the node was generated. This is useful for debugging and error reporting.

- **nodekind:** Enumeration defining the kind of node (e.g., `funcDefNK`, `idNK`, `declarNK`). This field determines the semantic interpretation of the node.
- **nodeAttr:** A structure or union that holds additional attributes depending on the node type (e.g., identifier name, literal value, operator type, etc.).

To properly construct and link nodes, two helper functions are used:

- `createNode(NodeKind kind)`: Allocates and initializes a new AST node of the specified kind. It sets the node's kind, initializes the child and sibling pointers to `NULL`, and stores the line number and any relevant attributes. This function is the entry point for dynamically constructing the tree during parsing.
- `addChild(TreeNode* parent, TreeNode* child)`: Adds a child node to a parent node. If the parent has no children yet, the child is directly assigned. Otherwise, the new child is appended to the end of the existing child list.
- `addSibling(TreeNode* node, TreeNode* sibling)`: Links a node as a sibling of the current node, inserting it at the end of the sibling list.

This node-based model provides a clear and extensible representation of the program's structure, enabling further stages such as semantic analysis and code generation.

Node Typing and Kinds

To represent different syntactic and semantic elements of the language, each AST node includes a node kind identifier, defined using the `NodeKind` enumeration. This categorization allows the compiler to distinguish between functions, control structures, declarations, expressions, and various language constructs.

The main categories of node kinds include:

- **Structural Nodes:** Represent control flow and block structure (e.g., `funcDefNK`, `blockNK`, `ifNK`, `forNK`, `whileNK`).
- **Declaration Nodes:** Represent variable and function declarations (e.g., `declarNK`, `parameterNK`, `typedefNK`, `structNK`).
- **Expression Nodes:** Represent operations, function calls, literals, and identifiers (e.g., `opNK`, `funcCallNK`, `idNK`, `numberNK`).

- **Modifiers and Attributes:** Represent qualifiers, storage classes, and data types (e.g., `qualifierNK`, `storageNK`, `typeNK`).
- **Array and Access Nodes:** Represent array usage and pointer/member access (e.g., `arrayNK`, `indexNK`, `memberAccessNK`).

To support fine-grained semantic distinction, other enumerations are used within the node's attributes:

- **Declark:** Specifies fundamental C types (e.g., `intK`, `floatK`, `charK`).
- **QualifierK:** Encodes type qualifiers like `const` and `volatile`.
- **StorageK:** Represents storage class specifiers such as `static` and `extern`.
- **opK:** Defines supported operators, including arithmetic, logical, bitwise, assignment, and unary/postfix operators.

This typing system ensures that each node in the AST is precisely defined and handled during semantic analysis and later compilation stages.

Helper Functions

In order to efficiently build and manipulate the Abstract Syntax Tree (AST), the compiler relies on a set of utility functions that manage the creation and structural linking of nodes. This subsection presents the core helper functions — `createNode`, `addChild`, and `addSibling` — along with illustrative diagrams that describe their internal logic and control flow.

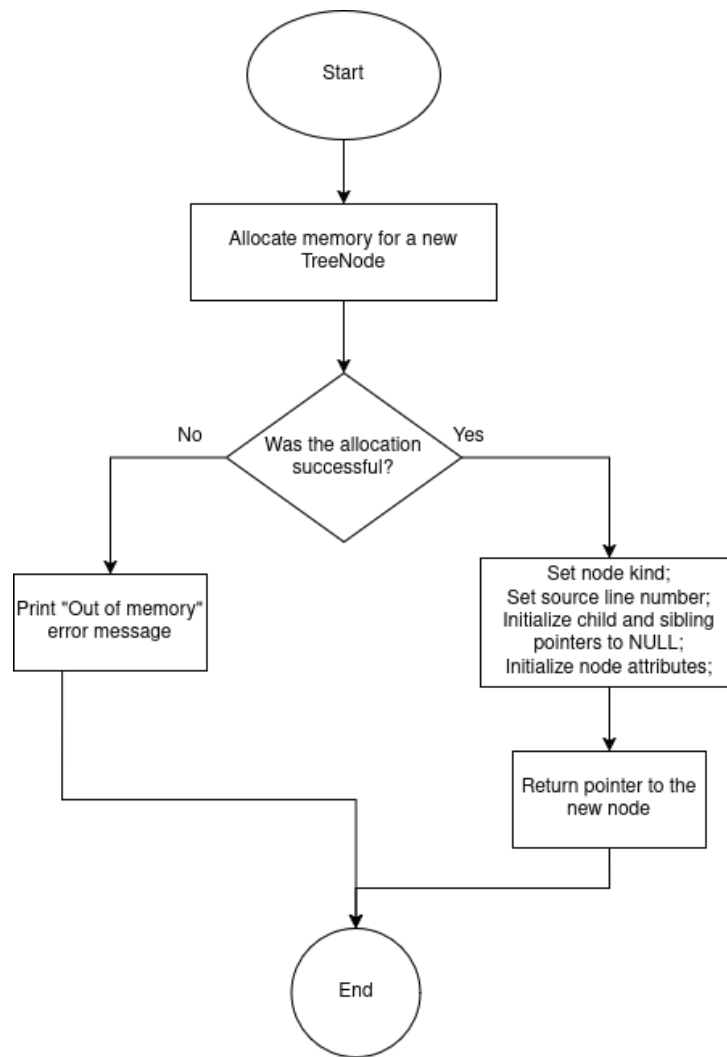


Figure 2: CreateNode() Flowchart

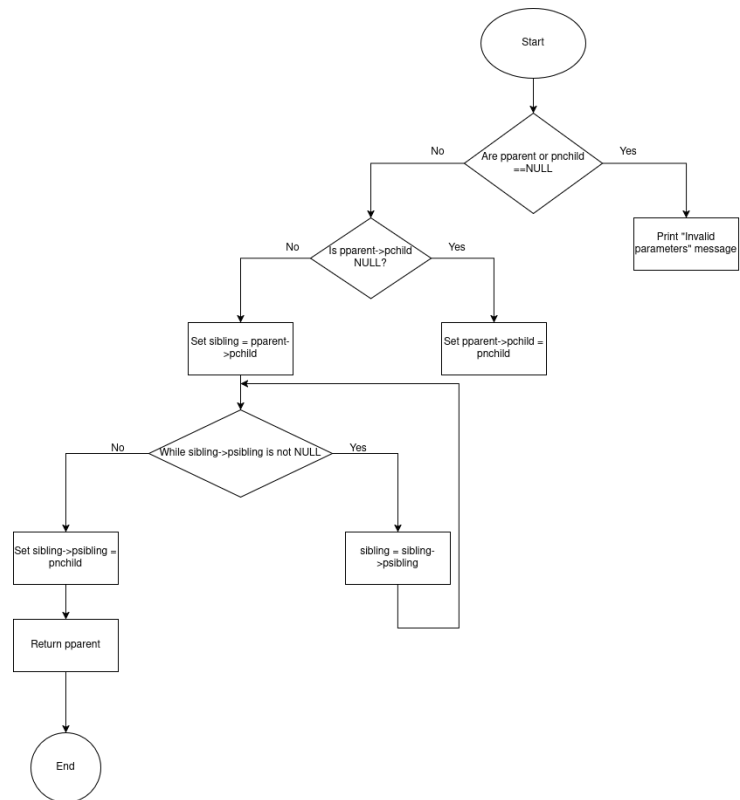


Figure 3: Addchild() Flowchart

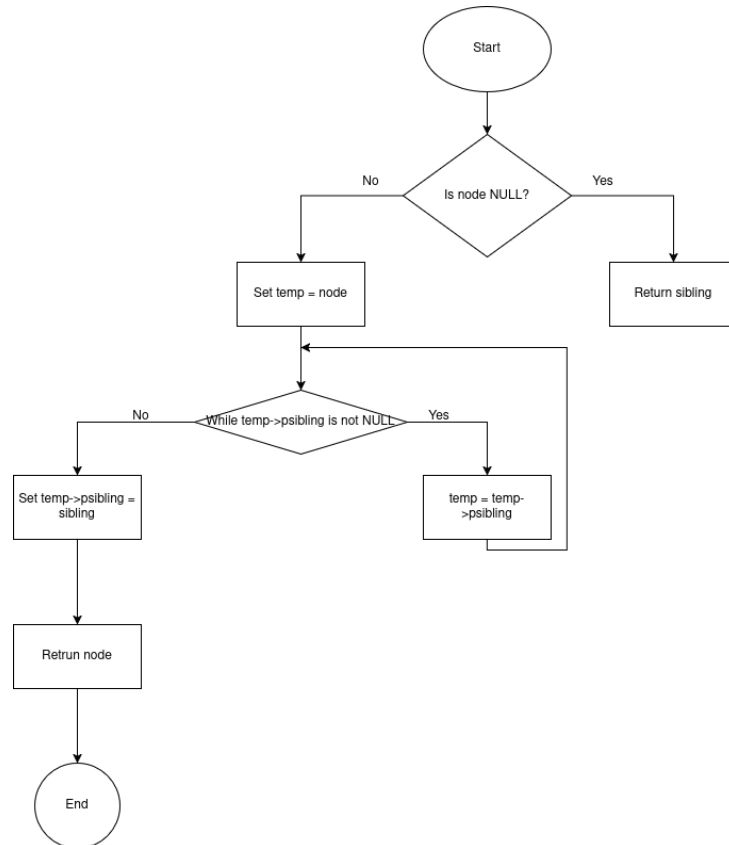


Figure 4: AddSiblings() Flowchart

Generic Tree Design

This section presents representative examples of generic Abstract Syntax Tree (AST) structures corresponding to typical language constructs handled by the compiler. Each tree illustrates how nodes are organized using the internal parent-child and sibling model. The diagrams cover common constructs such as function definitions, loops (for), conditional statements (if), variable initialization, constants, and function calls. These generic patterns serve as a foundation for understanding how source code is transformed into hierarchical, navigable tree structures during the parsing phase.

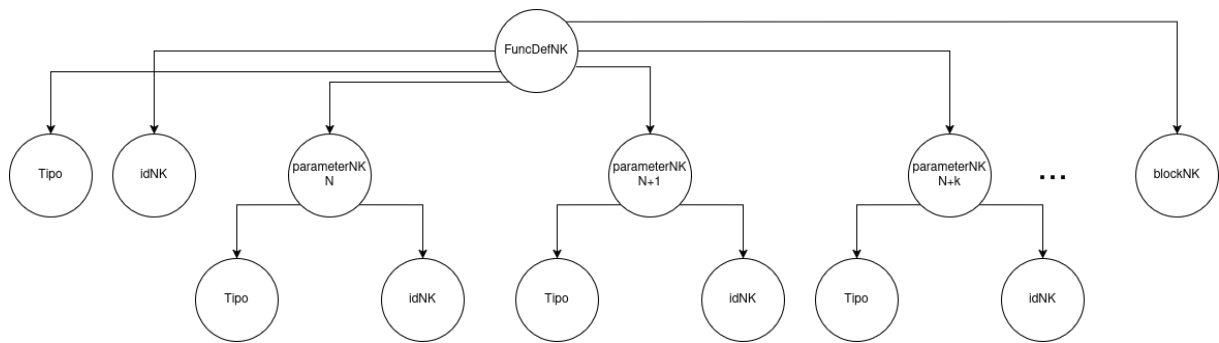


Figure 5: Function Definition AST

The AST starts with `FuncDefNK` (function), which includes the return type (`Tipo`), name (`idNK`), parameters (`Tipo + idNK` for each), and body (`blockNK`).

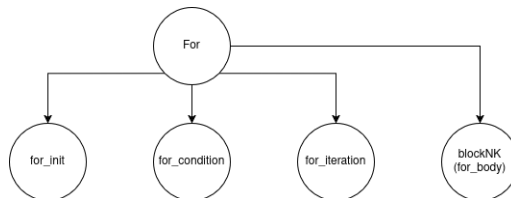


Figure 6: For Loop AST

The for loop is decomposed in the AST into initialization (`For_init`), condition (`for_condition`), iteration (`for_iteration`), and body (`blockNK`).

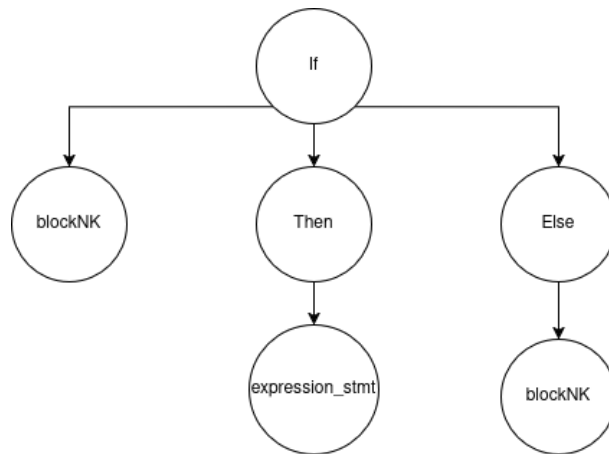


Figure 7: If Statement AST

The `if` statement in the AST has a root node `If` with children for the condition (`expression_stmt`), the `Then` block (executed if true), and optionally an `Else` block. `blockNK` represents the body of the blocks.

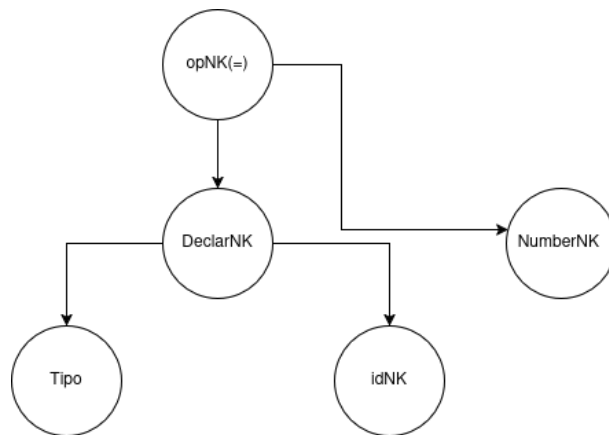


Figure 8: Variable Initialization AST

The AST shows a variable declaration (`DeclarNK`) with a type (`Tipo`), name (`idNK`), assignment operator (`opNK(=)`), and initial value (`NumberNK`). Used in compilers to represent declarations like `int x = 5;`.

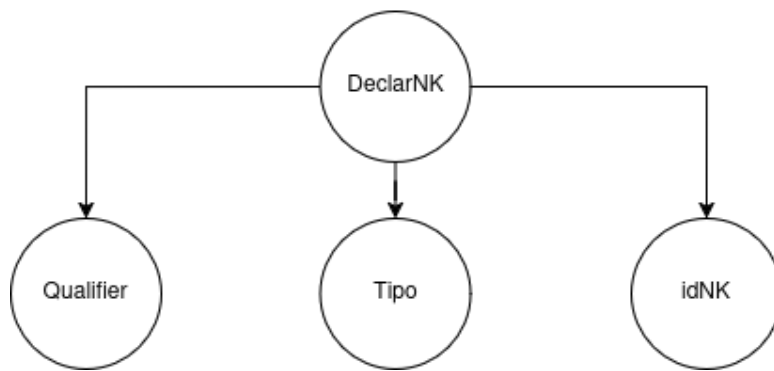


Figure 9: Const Variable AST

The AST represents a constant variable, with a qualifier (`const` or `volatile`), type (`Tipo`), and name (`IdNK`). Used for immutable declarations like `const int MAX = 100;`.

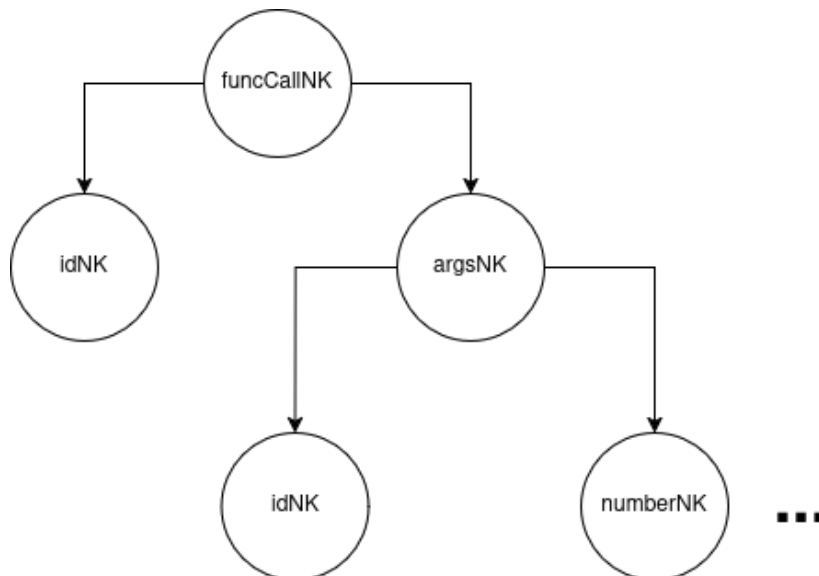


Figure 10: Function Call AST

The AST for a function call (`funcCallNK`) includes the function name (`idNK`) and its arguments (`argsNK`), which can be identifiers (`idNK`) or literals (`numberNK`).

Grammar-to-AST Mapping

This section describes how specific grammar rules defined in the parser are translated into corresponding Abstract Syntax Tree (AST) structures. The transformation from grammar to AST is a crucial step in bridging the gap between syntactic analysis and later phases such as semantic checks or code generation.

For each relevant language construct, we explain the corresponding grammar production and the

associated AST construction logic. This includes node creation, hierarchy definition (via parent-child and sibling relationships), and node attribute assignment. The goal is to ensure that each syntactic element is faithfully and consistently represented within the tree structure.

Variable Declaration

Grammar Rule:

```
declaration_specifiers ID ASSIGN NUMBER SEMICOLON
```

Description: This rule defines a variable declaration with initialization. The resulting AST is structured around an assignment node (opNK), with the left-hand side containing type and identifier, and the right-hand side being a numeric literal.

AST Structure:

```
opNK (=)
  DeclarNK
    typeNK (int)
    idNK (a)
  numberNK (5)
```

Function Definition

Grammar Rule:

```
declaration_specifiers ID OPEN_PAREN parameter_list CLOSE_PAREN OPEN_CURLY block_line
```

Description: This rule defines a function with a return type, name, parameters, and a block of statements. The AST uses funcDefNK as the root, with children representing the return type, function name, parameter list, and function body.

AST Structure:

```
funcDefNK
  typeNK (void)
  idNK (func)
  parameterNK
    typeNK (char)
    idNK (a)
  blockNK
  DeclarNK
```

```

    typeNK (int)
    idNK (b)
DeclarNK
    typeNK (double)
    idNK (c)
DeclarNK
    typeNK (float)
    idNK (d)

```

If Statement

Grammar Rule:

```
IF OPEN_PAREN expression_stmt CLOSE_PAREN OPEN_CURLY block_lines CLOSE_CURLY ELSE if_
```

Description: This rule represents an if-else statement. The AST is composed of an `ifNK` root node, a `thenNK` node containing the condition, and `blockNK` nodes representing the if and else branches.

AST Structure:

```

ifNK
    thenNK
        expression_stmt (e.g., numberNK (1))
    blockNK
        ... (statements inside if block)
    elseNK
    blockNK
        ... (statements inside else block)

```

Summary

The design of the Abstract Syntax Tree (AST) establishes a robust and extensible structure for representing the syntactic and semantic elements of the source code. Through a well-defined node architecture, categorized kinds, and a clear child-sibling hierarchy, the AST enables consistent and expressive modeling of language constructs. The supporting helper functions such as `createNode`, `addChild`, and `addSibling` provide a clean and modular mechanism for tree construction during parsing. This foundation ensures that the compiler can perform further analysis and transformations effectively, setting the stage for subsequent phases like semantic checks and code generation.

3.3 Semantic Analysis

In this phase, we describe the design decisions made for the implementation of the semantic analyzer. This includes the design of the symbol table and its integration with the abstract syntax tree (AST). The goal is to ensure efficient semantic validation while preserving correctness and supporting the requirements of our language model.

3.3.1 Symbol Table

Symbol Table Architecture

The symbol table is to be implemented as a hierarchical structure of hash tables, where each hash table represents a scope. This structure is chosen to reflect the lexical scoping rules of C, in which each function, block, or compound statement can introduce a new scope. To enable efficient traversal and scope resolution, each scope (symbol table) requires:

- A pointer to its **parent scope**, which allows symbol lookup to recurse upward through the lexical scope chain.
- A pointer to its **first child scope**, which represents the innermost scope nested within it (e.g., the body of a function or a block inside a loop).
- A pointer to its **next sibling scope**, which enables traversal of sibling scopes at the same nesting level (e.g., separate branches of an if-else).

This organization forms a tree of symbol tables that mirrors the structure of the source code, allowing semantic rules involving visibility and shadowing to be enforced accurately.

Structure of Symbol Table Entries

Each entry in a symbol table corresponds to a declared identifier, such as a variable, function, or parameter. Each entry contains several attributes that are critical to semantic analysis:

- **Name**: the string identifier of the symbol.
- **Kind**: the kind of symbol (e.g., variable, function, parameter, array).
- **Declared type**: the semantic type of the symbol (e.g., `int`, `float`, `char`).

- **Storage class and qualifiers:** optional attributes such as `static`, `extern`, or `const`.
- **IsPointer:** whether the symbol is a pointer.
- **IsArray:** whether the symbol is an array.
- **IsUnsigned:** whether the symbol is an unsigned type.
- **IsDefined:** whether the symbol is a full definition (as in a function) or just a declaration.
- **Memory location (optional):** memory offset for code generation, if needed.
- **Line number list:** the source lines where the identifier appears, for error reporting and analysis.

These fields support the enforcement of many semantic rules, such as type checking, multiple declarations, and incompatible redefinitions.

Why Use a Hash Table for Scopes?

Hash tables are used for symbol lookup within each scope due to their average-case constant time complexity, which ensures rapid insertion and lookup of identifiers. Since semantic analysis must perform frequent queries on the symbol table — such as when checking for duplicates, resolving variable types, or determining function signatures — hash tables provide a highly efficient mechanism for managing symbols.

Additionally, since symbol names are strings and string comparisons are costly, hashing enables a more scalable solution than a linear or tree-based search structure. With a well-designed hash function and reasonable table resizing strategies, collisions are minimal and performance remains optimal even in large programs.

Linking AST Nodes to Symbol Tables

Each node in the abstract syntax tree (`TreeNode`) should have a pointer to the symbol table (scope) in which it appears. This design decision provides several advantages:

- It simplifies type checking and semantic validation by allowing immediate access to the relevant scope context for any node.
- It ensures that lookups for variable types or function symbols are always performed in the correct lexical scope.

- It facilitates scope-aware traversal during both symbol insertion and semantic checking, especially in languages with block-level scope like C.
- It enables diagnostic tools (e.g., warnings for unused variables or scope visualization) to easily correlate AST structure with symbol visibility.

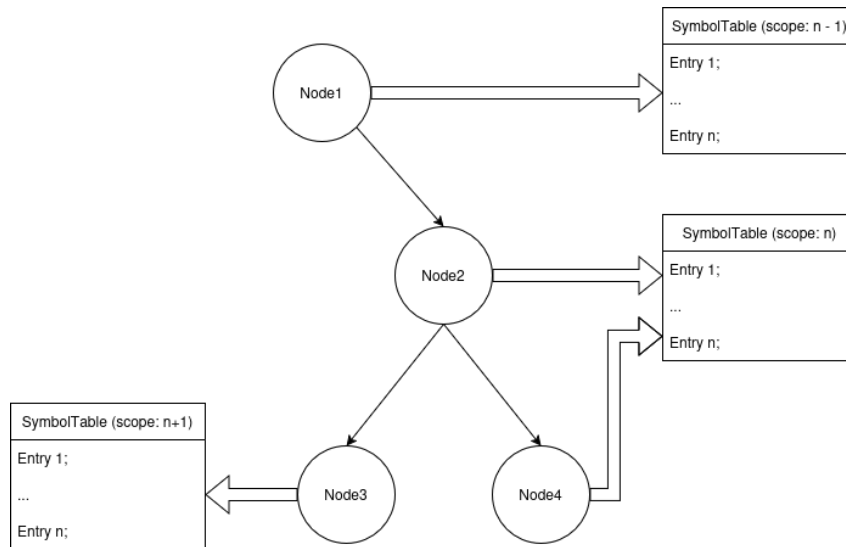


Figure 11: Simple representation of tree node pointers to each node's scope table.

This explicit association between AST nodes and their corresponding scopes makes the semantic analyzer robust and context-aware. It avoids the need to recompute or infer scope during later traversal phases, improving both clarity and performance.

Scope Nesting and Management

When a new scope is encountered — such as entering a function, a compound statement block, or a control-flow construct (if, while, for, etc.) — a new symbol table should be created. This table should then be linked to the current scope as its child, and the semantic analyzer must update the current scope pointer to refer to it. Once the block is processed, the analyzer restores the parent scope.

This nesting model ensures that identifiers declared in inner blocks do not pollute outer scopes, and that outer symbols are correctly shadowed by local declarations.

Using a simple example:

```

1 int x;
2
3 void f1(){

```

```

4      ...
5  }
6
7  void f2(){
8      ...
9  }

```

The diagram for the symbol tables would be something similar to Figure 12:

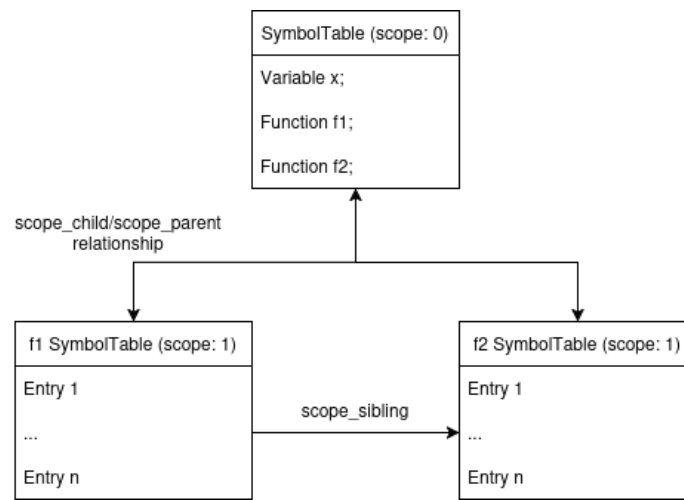


Figure 12: Symbol tables created for the example, their scopes and relationship to each other.

Design of Utility Functions for Symbol Table Management

To support the construction and consultation of symbol tables during semantic analysis, several utility functions have been designed. Each function is made to address a specific task regarding the symbol tables. Below is a detailed description of each utility function and its purpose within the symbol table infrastructure.

createTable

The `createTable` function is responsible for instantiating a new symbol table, representing a new lexical scope. The function allocates memory for a new symbol table, initializes its internal parameters, and links it to the parent table. If the parent scope already has child scopes, the new table is appended as a sibling in the scope hierarchy.

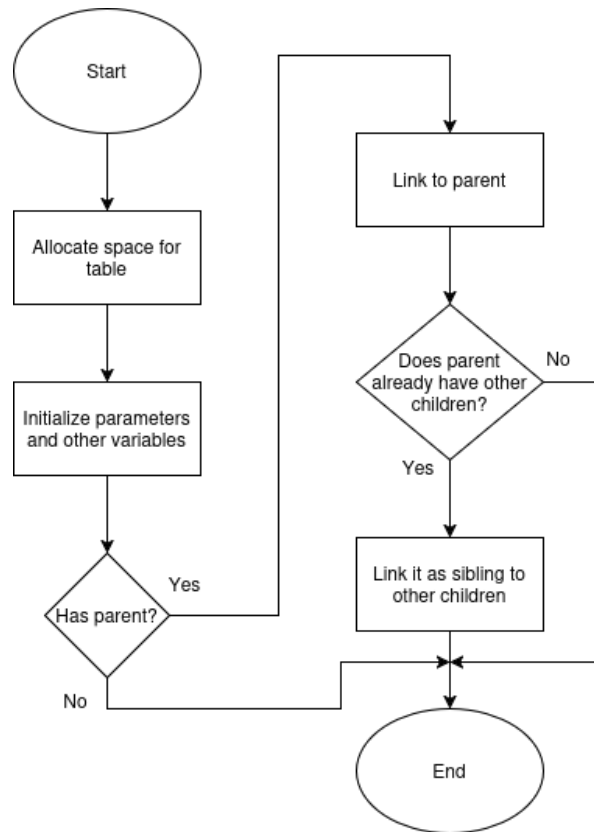


Figure 13: Flowchart for the createTable function.

This structural design enables traversal and lookup based on the lexical nesting of blocks, which is fundamental for correctly applying nearest-scope rules during semantic resolution.

st_insert

The `st_insert` function is responsible for registering new symbols within a specific scope's symbol table. When a variable, function, or parameter is declared or defined, this function is called to ensure the symbol is recorded for future semantic checks. Should the symbol already exist in the current scope, the function does not override it but instead appends the new line of occurrence to the record's line history, allowing the compiler to track multiple references.

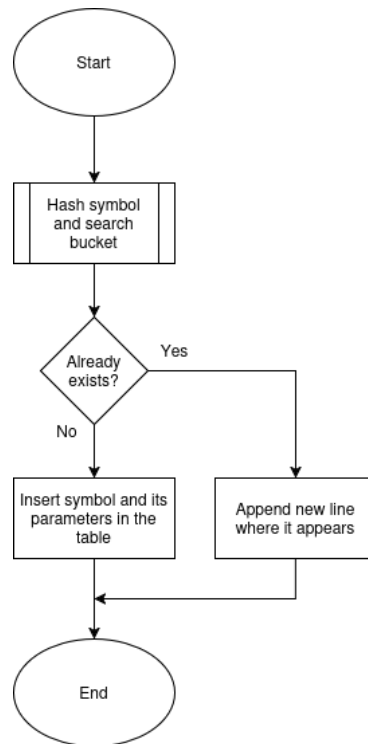


Figure 14: Flowchart for the `st_insert` function.

`st_lookup`

The `st_lookup` function recursively searches for a symbol starting from a given scope and progressing outward through parent scopes. This mirrors ANSI C's lexical scoping rules, where identifiers declared in inner blocks shadow those in outer blocks, but identifiers not found in a local block may still be resolved from parent scopes.

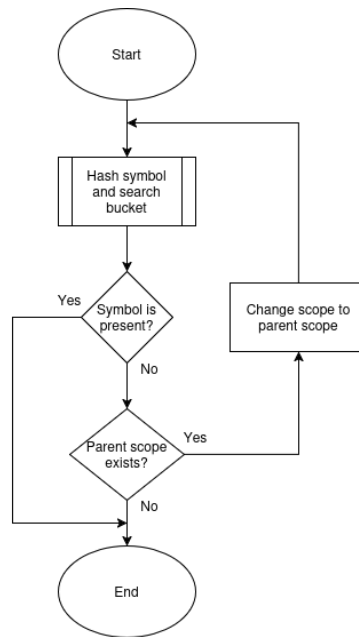


Figure 15: Flowchart for the st_lookup function.

st_lookupLocal

The `st_lookupLocal` function performs a non-recursive search for a symbol within the current scope only. It is typically used during insertion to check for redefinitions or duplicate declarations within the same lexical block.

It hashes the symbol name and scans only the current table's bucket for a match. If none is found, the symbol is assumed to be new in that scope.

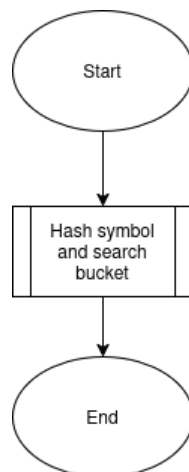


Figure 16: Flowchart for the st_lookupLocal function.

3.3.2 Type Checking

In this section, we present the design considerations and implementation details for the type checking phase of the semantic analyzer. Type checking plays a crucial role in ensuring that operations, assignments, and function calls are semantically valid, according to the rules of the language.

Role of Type Checking in the Semantic Analyzer

Type checking operates on the abstract syntax tree (AST) after the symbol table has been fully populated. At this point, each identifier is associated with a corresponding symbol table entry, and every node in the AST is linked to its lexical scope. This contextual information allows type checking to verify expressions and declarations accurately.

Type checking is designed to perform the following key tasks:

- Ensure that operands of the expressions are compatible with the operators applied to them.
- Validate that function calls match declared signatures in number and type of arguments.
- Insist on consistency of the return type in the functions.
- Detect invalid type conversions or operations on incompatible types.
- Verify the appropriate use of control statements like `return`, `break`, and `continue`.

Traversal Strategy for Type Checking

Type checking is implemented as a post-order traversal of the AST, using a recursive function `checkNode(TreeNode* t)`. For each node, the function first processes its children (which represent subexpressions or substatements), then applies specific type rules depending on the node's kind (expression, statement, declaration, etc.).

This bottom-up approach ensures that the types of subcomponents are known before evaluating the parent node.

Expression Type Inference and Validation

For binary and unary operators (e.g., `+`, `-`, `*`, `/`, `<`, `==`), the types of operands are checked to ensure compatibility. If types mismatch, an error is reported.

Although arithmetic and logical operators are primarily intended for numeric types, this implementation also accounts for cases involving mixed types and implicit promotions.

- Operations between values of the same numeric type (`int`, `float`, `double`) are allowed without restriction.
- Certain mixed-type operations are accepted, particularly when one operand is a literal or constant. For example, a variable of type `char` can be incremented with an integer constant (`char t = 0; t = t + 6;`), which is valid due to implicit promotion.
- However, operations involving different non-literal types (e.g., `char` and `int`, or `int` and `float`) require special handling and may result in a type error unless an explicit or implicit conversion is supported.
- Incompatible operations (e.g., involving `void` or function types) are disallowed and flagged as errors.

These checks are explicitly enforced to prevent undefined behavior and ensure that all expressions follow the language's typing rules.

- Arithmetic operators (`+`, `-`, `*`, `/`) require numeric operands.
- Logical and comparison operators (`<`, `>`, `==`) require operands of comparable types.
- The result type is inferred based on operand types and the semantics of the operator.

Implicit conversions (such as promoting an `int` to a `float`) are supported in some contexts, but are handled explicitly by the type checker to avoid unintended semantics.

Assignment and Variable Use

Assignment statements are validated to ensure that:

- The left-hand side (LHS) is a valid *l-value* (i.e., a variable or dereferenced pointer).
- The type of the RHS expression is assignable to the LHS type.
- Incompatible assignments (e.g., assigning a `float` to an `int` without conversion) raise errors.

Each variable usage is linked to its symbol entry, and its type is retrieved during analysis.

Function Call Validation

Function calls undergo several checks:

- The function being called must be declared and its symbol must be of kind *function*.
- The number of arguments provided must match the number of parameters declared.
- The types of each argument must match the expected parameter types.
- The resulting type of the function call is the function's return type, and is assigned to the expression node.

Function prototypes stored in the symbol table are used to perform this verification.

Return Statement Checks

For each `return` statement, the returned expression is validated against the return type declared in the enclosing function:

- If the function returns `void`, any expression in the `return` statement is an error.
- If the function expects a value, the type of the returned expression must match (or be implicitly convertible to) the function's return type.

Each function's return type is retrieved from its symbol table entry during the check.

Control Flow Statements: `goto`

`goto` must reference a valid label. If not, a semantic error is issued.

Error Reporting and Robustness

Type mismatches and illegal usages are reported with the following information:

- A clear description of the error, including the expected and actual types
- The context of the error, such as the operation, function, or expression where the error occurred

The checker does not immediately abort the analysis upon encountering errors; instead, it continues processing to allow the detection of multiple semantic issues in a single pass.

3.4 Code Generation

The Code Generation phase is responsible for translating the high-level abstract representation of the source program, expressed in the Abstract Syntax Tree (AST), into low-level target code. In this compiler, the backend is designed to produce assembly code tailored for the Intel 8051 Instruction Set Architecture (ISA), a widely used microcontroller platform.

The design of the code generation module involves mapping each relevant AST node into its corresponding instruction sequence, ensuring that data types, control flow, and memory layout are preserved correctly in the target code. This section outlines the overall strategy adopted for converting different language constructs—including expressions, declarations, conditionals, and function calls—into valid and efficient 8051 assembly instructions.

3.4.1 Architecture and Role of the Back-End

The back-end architecture was designed with a focus on simplicity and modularity—essential characteristics in embedded systems development environments. Its main function is to transform the abstract syntax tree (AST), already analyzed and annotated by the semantic phase, together with the symbol table, into assembly code specific to the target architecture.

The code generator is composed of modularly organized functions. Each module is responsible for a well-defined task: generating instructions for expressions, flow control, function calls, and more. This structure facilitates code readability and maintenance, while also allowing for future expansion if new functionalities or instructions need to be added.

Unlike traditional compilers that use an intermediate textual representation (such as three-address code), the back-end in this project operates directly on the AST. The tree is traversed recursively, and each node is analyzed according to its type and context. Semantic information—such as data types, scopes, sizes, and variable locations—is already embedded within the AST nodes and is used directly by the generator. This eliminates the need to convert the AST into another intermediate form, reducing implementation complexity.

The data flow between the compilation phases is therefore straightforward and sequential: the AST produced by the front-end is passed to the back-end as input without any further modifications. From there, the generator emits assembly instructions, which are written to an output .a51 file.

3.4.2 Symbol Table Integration

The symbol table plays a critical role in the code generation phase. It maintains structured information about all declared identifiers, including their names, types, scopes, and memory bindings. During code generation, the generator queries the symbol table to resolve variables and function references, determine storage requirements, and apply the correct memory access patterns. This integration ensures type-correct assembly generation and enforces scoping rules defined earlier in the compilation process. Furthermore, the symbol table supports consistent register allocation, label naming, and function call conventions, enabling accurate and maintainable translation from the abstract syntax tree to target assembly code.

3.4.3 Recursive Tree Traversal and Node Categorization

Once the Abstract Syntax Tree (AST) and the symbol table are available, the code generation phase begins by traversing the AST recursively. A crucial first step is the classification of each node into one of three broad categories: statements, expressions, and functions. This categorization is essential for managing the recursive descent through the tree, as each category triggers distinct code generation logic that corresponds to the semantics of the source language (C) and its mapping to the target architecture (8051).

Statements include constructs such as `if`, `while`, `for`, and `switch-case` (represented by nodes like `ifNK`, `whileNK`, `forNK`, `switchNK`, and `caseNK`). These nodes typically control flow and do not directly produce values, but instead dictate the structure and sequencing of the generated assembly. Their code generation involves condition evaluation, label creation, and appropriate branching instructions.

Expressions, by contrast, such as `numberNK`, `idNK`, `opNK`, and `blockNK`, are value-producing and generally occur on the right-hand side of assignments, within conditionals, or as arguments to functions. These nodes require evaluation and temporary register allocation, and their recursive handling allows the decomposition of complex expressions into manageable instructions.

Function-related nodes — `funcDefNK`, `funcDeclNK`, `funcCallNK`, `parameterNK`, `argsNK`, and `returnNK` — have their own dedicated logic because they involve context management (e.g., stack frame setup, parameter passing, return value handling, and local scope management). Generating code for functions requires both preparation (prologue generation) and cleanup (epilogue), and the recursive traversal

ensures that all internal structures (body, parameters, and possible nested calls) are visited and handled properly.

The recursive structure of the `linegen()` function reflects this organization: depending on the node type, it dispatches to the corresponding generator (`genStmt`, `genExp`, or `genFunc`), and then continues traversal via the sibling pointer (`psibling`). This recursive approach ensures depth-first coverage of the AST while maintaining modular code generation — each category handles its semantics independently and composes cleanly with the others. This separation also simplifies debugging and allows future extensions (e.g., new statement types or expression forms) to be integrated with minimal disruption.

3.4.4 Memory Allocation and Variable Management

Global variables, as well as local variables, are stored directly in RAM. Although the 8051 architecture provides a limited set of register banks (Bank 0 to Bank 3), relying heavily on these for variable storage introduces significant complexity. An initial strategy of using separate register banks for different purposes — for instance, Bank 0 for temporaries, Bank 1 for parameters, and Bank 2 for local variables — quickly proves unfeasible. This approach would require frequent and costly switching between register banks, leading to inefficient code, increased instruction count, and potential bugs due to context inconsistency during nested function calls or expression evaluations.

To avoid these complications and ensure consistent access to variables, all local and global variables are allocated in the data segment (RAM), under the DSEG section. This design simplifies register usage, avoids unnecessary bank switching, and centralizes variable storage. However, storing all variables in the same memory space introduces a new challenge: name collisions, particularly for local variables with the same identifier across different functions.

To solve this, each local variable is renamed using a convention that prefixes the function name, forming identifiers in the pattern: `functionName_variableName`. For example, a local variable `count` inside function `main` becomes `main_count` in the generated code. This naming convention ensures uniqueness across the entire RAM space while maintaining readability and traceability of variable origins.

3.4.5 Stack-Based Parameter Passing and Calling Conventions

In the case of parameter passing between functions, the values of each argument are dynamically allocated on the stack, and then stored in the first registers bank, starting by R2, through the accumulator. The code generator uses explicit control of the stack pointer to determine the exact offset of each parameter. This strategy allows functions to be re entrant and supports nested or recursive calls without risk of variable or parameter overwriting, which would happen if fixed memory locations or registers were reused across function calls. Since the 8051 has a small number of general-purpose registers and no built-in call stack for parameters, using the stack is the most scalable and safe method for parameter management.

3.4.6 Register Usage for Temporary Data and Type Handling

This overall strategy — placing variables in RAM, using the stack for parameter passing, and uniquely renaming locals — strikes a balance between simplicity, efficiency, and correctness. It avoids the complexity of register bank management and ensures that function execution remains isolated and predictable, which is critical in the constrained and deterministic environment of embedded systems like the 8051.

For variables of type char or numeric literals within 8-bit range, values are passed directly via the accumulator. However, for int variables or numbers greater than 255 (the maximum value for a char), which require 16-bit representation, the values are split and temporarily stored in two registers from the first register bank, specifically, R2 and R3. The accumulator alone cannot hold 16-bit values, so R2 and R3 are used as temporary holders. These registers are not preserved across calls and are reserved solely for intermediate arithmetic processing.

3.4.7 Translation of Control Structures

When an operation is identified, the types of its child nodes—whether char or int—must first be determined. For char operands, values are temporarily held in the accumulator and register R0. For int operands, the first value is stored in R2 and R3, while the second is stored in R4 and R5. The corresponding assembly instructions are then generated using these registers. In cases where the operation produces results that do not fit in 16 bits, the result is truncated to store only the least significant 16 bits (LSB).

3.4.8 Memory management

Based on the previously described memory management strategy, the following diagram illustrates how different types of data—including global variables, local variables, function parameters, and code—are mapped to the appropriate memory regions of the 8051 microcontroller.

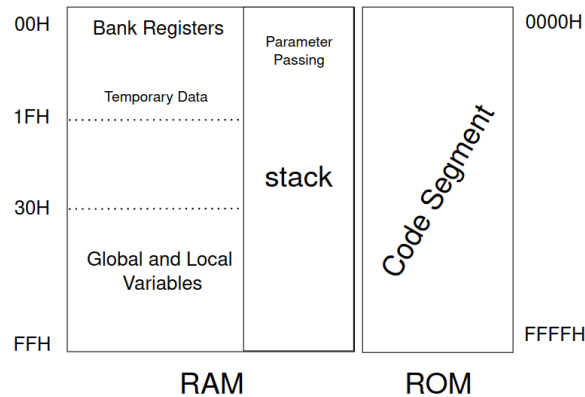


Figure 17: Memory management for code generation

3.4.9 Function Code Generation and Return Semantics

Control structures such as if, while, and for are mapped to conditional jump instructions and dynamically generated labels (e.g., IF_Lx, FOR_START_x, _WHILE_END_x). These labels simulate structured control flow using unconditional and conditional jumps. Since assembly lacks high-level constructs, all control structures must be manually translated into explicit branching logic. The condition expression is evaluated first, and its boolean result is stored in the accumulator. An if-else statement, for example, generates two labels: one for the "then" branch, when the condition is true, and one for the "else" branch, when it is false. Loops like while and for are converted into jump-based cycles with explicit condition testing: the condition is evaluated in each iteration, and the body is executed if true, or the loop is exited if false.

In function definitions, the return type and identifier are first analyzed to extract the function name. Parameters are then generated, if present. The AST is traversed node by node to generate the complete function body. In function calls, parameters are pushed onto the stack before the call is issued. The called function receives parameters from the stack and executes its body until reaching the return node. If the return type is char, the result is passed back via the accumulator. If the return type is int, the result is returned using the accumulator and register R2.

Chapter 4

Implementation

This chapter details the practical realization of the compiler components described in the previous design phase. It focuses on how the key structures and functionalities—such as lexical analysis, syntax parsing, AST construction, and symbol handling—were translated into working code.

The implementation is modular, reflecting the architecture outlined in the system overview. Each compiler stage is implemented in its own module or file, promoting clarity, maintainability, and ease of debugging. This includes the lexer (built with Flex), the parser (built with Bison), and the logic for building and traversing the Abstract Syntax Tree (AST).

Throughout this chapter, the main implementation decisions, data structures, and logic flows are presented, highlighting the alignment between the grammar rules and the tree-building operations. Where relevant, snippets of representative code and diagrams are used to illustrate the internal mechanics and flow of information.

4.1 Lexical Analysis(Lexer)

The lexical analysis phase is implemented using Flex, which generates a scanner based on regular expressions defined in the lexer file. This scanner reads the input source code and breaks it into a stream of tokens such as keywords, identifiers, literals, operators, and punctuation. Each token is then passed to the parser along with relevant metadata (e.g., line number or value). The lexer also handles whitespace, comments, and escape sequences, ensuring that only syntactically meaningful components are forwarded to the next stage. This phase forms the foundation for correct syntactic and semantic analysis.

4.1.1 Implementation of Scanner Rules

The lexical analyzer (scanner) was implemented in Flex, organized into several rule blocks that enable the recognition of all required elements of the C language and 8051 extensions. Below, the main groups of

implemented rules are presented, with real examples, code excerpts, and explanations of their utility.

- **Digits, Letters, and Identifiers**

The basis for recognizing most elements in C lies in the definition of digits and letters. From these fundamental definitions, the rule for identifiers is built. A valid identifier may start with a letter (uppercase or lowercase), and be followed by any combination of letters or numbers.

Listing 4.1: Definition of digits, letters, and identifiers

```
1 _digit      [0-9]
2 _letter     [a-zA-Z_]
3 _identifier {_letter}{(_letter)|{_number}}*
```

An identifier can start with a letter and be followed by letters or digits. Valid examples: `main`, `contador2`.

- **Number Recognition**

The scanner is capable of interpreting numbers in different numeric bases, a crucial feature for code-writing flexibility. Decimal, octal, binary, and hexadecimal bases are supported, with specific patterns for each one.

Conversion of these numbers to their corresponding integer values is done using the `strtol` function, with the appropriate base specified. The value is stored in the union `yylval.val`, and the `NUMBER` token is returned to the parser.

Listing 4.2: Rules for numbers in various bases

```
1 _binary_number 0[bB][01]+
2 _octal_number  0[0-7]+
3 _hex_number     0[xX][0-9a-fA-F]+
4 _number         (0|[1-9][0-9]*)
```

These patterns allow detection of decimal, octal, binary, and hexadecimal numbers. `0b1010`, `077`, `0x1A3`, `42` are all recognized.

The scanner converts to integer value with:

Listing 4.3: Number conversion

```
1 {_hex_number} { yylval.val = strtol(ytext, NULL, 16); return
    NUMBER; }
```

```

2 {_octal_number} { yylval.val = strtol(yytext, NULL, 8); return
   NUMBER; }
3 {_binary_number} { yylval.val = strtol(yytext + 2, NULL, 2);
   return NUMBER; }
4 {_number} { yylval.val = strtol(yytext, NULL, 10); return NUMBER
   ; }

```

• Floating Point Numbers (Float)

In addition to integers, the scanner also detects floating-point numbers, which represent real values. The `_float_number` rule enables recognition of different float number formats.

Listing 4.4: Float recognition

```

1 _float_number    {_digit}*"."{_digit}+

```

Valid examples: 3.1415, .5, 0.01.

After recognition, the string is converted to a floating-point value using `strtod` and stored in `yylval.fval`, returning the `FLOAT_NUMBER` token.

```

1 {_float_number} { yylval.fval = strtod(yytext, NULL); return
   FLOAT_NUMBER; }

```

• Strings and Characters

The ability to handle string and character literals is fundamental for any C compiler. The implemented rules support escape sequences, allowing representation of special or non-printable characters inside strings and character literals.

Listing 4.5: Rules for strings and characters

```

1 _string [""]([^\n]|\\.|\\n)*[""]
2 _charV '([^\n]|\\[abfnrtv\\'"]|\\[0-7]{1,3}|\\x[0-9a-fA-F]+)'

```

They accept escapes such as `'\n'`, `'\x41'`, `"abc \t123"`.

The code associated with these rules is responsible for extracting and converting the value of the string or character, properly handling escape sequences.

• C and 8051 Language Keywords

All reserved words in C, as well as the specific extensions for the 8051 microcontroller, are carefully defined as separate rules in the scanner. Each keyword has its own rule, enabling precise recognition and assignment of a unique token for each one.

Listing 4.6: Keyword example

```
1 _auto "auto"  
2 _break "break"  
3 _return "return"  
4 _sfr "sfr"  
5 ...
```

When the scanner finds one of these reserved words, it returns the corresponding token to the parser.

```
1 {_auto} { return AUTO; }  
2 {_break} { return BREAK; }  
3 {_sfr} { return SFR; }
```

Recognized examples: if, while, return, sfr.

- **Preprocessor Directives**

Preprocessor directives are instructions that the C preprocessor executes before actual compilation. The scanner has rules to identify these directives, allowing the compiler to handle them appropriately.

Listing 4.7: Preprocessor directives

```
1 _preprocess_define "#define"  
2 _preprocess_include "#include"
```

For each recognized directive, a specific token is returned, informing the parser of the presence of a directive.

```
1 {_preprocess_define} { return PREPROCESS_DEFINE; }
```

- **Operators and Symbols**

All operators (arithmetic, logical, assignment, etc.) and symbols (such as parentheses, braces, semicolons) are explicitly defined in the scanner. This approach ensures that every fundamental syntactic element is correctly identified.

Listing 4.8: Rules for operators and symbols

```

1  _plus      "+"
2  _inc       "++"
3  _minus     "-"
4  _dec       "--"
5  _assign    "="
6  _assign_plus "+="
7  _equal     "=="
8  _semicolon ";"
9  _open_paren "("
10 _close_paren ")"
11 ...

```

Each of these patterns returns a dedicated token, which will be used by the parser to build the syntax tree. Recognized examples: +, ++, =, ==, ;, (.

- **Single-line and Multi-line Comments**

Comments are parts of the code that must be ignored by the compiler. The scanner is configured to detect and skip single-line comments (//) and multi-line comments (/* ... */), ensuring that no unnecessary tokens are generated.

Listing 4.9: Comment recognition

```

1  _line_comment  "//".*
2  _multiline_comment_start  "/*"
3  _multiline_comment_end    "*/"

```

For single-line comments, the scanner reads until it finds a newline or end of file. For multi-line comments, a more complex logic (usually in an exclusive lexer state) is used to consume all characters until the closing sequence is found.

```

1  {_line_comment} { int c; while ((c = input()) != '\n' && c != 0)
    ; }
2  {_multiline_comment_start} { ... }

```

Ignored examples: // comment, /* block */.

- **Whitespace and Newlines**

Whitespace (spaces and tabs) and newlines are important for code readability but do not contribute to its semantics. The scanner is configured to ignore these elements, producing no tokens for them.

Listing 4.10: Ignoring whitespace

```
1 _white_space    [ \t]+
2 _newline        \n
```

Ignored without producing tokens.

- **Invalid Sequences**

To ensure the robustness of the analyzer, it is essential that the scanner can identify and report invalid sequences that do not correspond to any valid token pattern. A common example is an identifier that starts with a digit, which is prohibited in C.

Listing 4.11: Invalid identifier

```
1 _invalid_sequence ([0-9]+{_letter}[a-zA-Z0-9_]*)
```

When an invalid sequence is detected, the scanner executes an action to print an error message, alerting the user to the issue.

- **End of File and Unrecognized Tokens**

The last two rules are crucial for handling control and error situations. The `<<EOF>>` rule is triggered when the scanner reaches the end of the input file, returning the `TOK_EOF` token to signal the parser that lexical analysis has finished. The dot rule (`.`) acts as a "catch-all" for any character that does not match any of the previous rules. This is useful for detecting unrecognized tokens and reporting them as errors, preventing the parser from receiving unexpected input.

Listing 4.12: End of file and unknown tokens

```
1 <<EOF>> { return TOK_EOF; }
2 . { printf("Unexpected_token:%s\n", yytext); }
```

These rules cover the entire spectrum of tokens accepted and rejected by the scanner, from reserved words, operators, numbers in any base, valid identifiers, strings/characters, to comments and whitespace. The goal is to ensure a robust, efficient lexical analysis, able to quickly distinguish valid and invalid situations in the source code.

4.2 Syntactic Analysis(Parser)

At this stage of the project, the grammar was developed to enable the compiler to recognize a subset of C, with 8051 extensions, and to build the corresponding Abstract Syntax Tree (AST). The `parser.y` file defines all the syntactic rules, which closely follow the structure of the language, with several alternatives to cover the multiple valid forms of each construct.

4.2.1 Parser - Grammar Rules

Program Structure

The entry point of the parser is the `program` rule, which encapsulates the entire source code, ensuring that the program ends with the end-of-file token (`TOK_EOF`).

Listing 4.13: `program` rule

```
1 program : code_lines TOK_EOF
```

The non-terminal `code_lines` represents a sequence of top-level code lines, allowing a program to be composed of one or multiple lines.

Listing 4.14: `code_lines` rule

```
1 code_lines : code_line
2             | code_lines code_line
```

Each `code_line` is comprehensive, and may correspond to a wide range of language constructs, from function definitions and prototypes to declarations, preprocessor directives, and different types of statements and blocks. This rule is crucial for the parser's flexibility.

Listing 4.15: `code_line` rule

```
1 code_line : function_definition
2           | function_prototype
3           | declaration
4           | define_directive
5           | expression_stmt
6           | while_stmt
7           | if_stmt
8           | for_stmt
```

```

9      | switch_stmt
10     | continue_stmt
11     | break_stmt
12     | goto_stmt
13     | return_stmt
14     | error_message
15     | line_info
16     | pragma_body

```

A program may include a main function, a #define directive, and an array declaration, as illustrated below:

```

1 int main() { return 0; }
2 #define MAX 100
3 unsigned char arr[5];

```

Functions: Definition and Prototypes

The function_definition rule is responsible for recognizing all valid forms of function definitions, whether functions have no parameters, a parameter list, or return pointers. The flexibility of this rule allows a wide range of function signatures to be parsed.

```

1 function_definition :
2     declaration_specifiers ID OPEN_PAREN parameter_list CLOSE_PAREN
3     OPEN_CURLY block_lines CLOSE_CURLY
4     | declaration_specifiers ID OPEN_PAREN CLOSE_PAREN OPEN_CURLY
5     block_lines CLOSE_CURLY
6     | declaration_specifiers pointer_opt ID OPEN_PAREN CLOSE_PAREN
7     OPEN_CURLY block_lines CLOSE_CURLY
8     | declaration_specifiers pointer_opt ID OPEN_PAREN parameter_list
9     CLOSE_PAREN OPEN_CURLY block_lines CLOSE_CURLY

```

Example:

```

1 void foo(int a, int b) { ... }
2 int *bar() { ... }

```

The `function_prototype` rule specifically handles function prototypes, which are declarations without the function body, ending with a semicolon. These declarations are essential for type checking and linking in C.

```
1 function_prototype : declaration_specifiers ID OPEN_PAREN
    parameter_list CLOSE_PAREN SEMICOLON
2                   | declaration_specifiers ID OPEN_PAREN CLOSE_PAREN
    SEMICOLON
```

Example:

```
1 void foo(int a, int b);
```

Declarations: Variables, Arrays, and Pointers

The `declaration` rule is one of the most extensive, as it encompasses a wide variety of declarations, from simple and initialized variables to arrays (with or without initialization), pointers, structures (`struct`), unions (`union`), enumerations (`enum`), and type definitions (`typedef`). Each variant is carefully modeled to ensure correct interpretation.

```
1 declaration : declaration_specifiers ID SEMICOLON
2             | declaration_specifiers ID ASSIGN logical_exp SEMICOLON
3             | declaration_specifiers ID OPEN_SQUARE NUMBER
    CLOSE_SQUARE SEMICOLON
4             | declaration_specifiers pointer_opt ID SEMICOLON
5             | ...
```

Examples of Variants:

- Initialized arrays, where values are defined at declaration time:

```
1 int arr[3] = {1, 2, 3};
```

- Pointers, which store memory addresses:

```
1 char *str;
2 float *vet;
```

- Structures and unions, which allow aggregation of data of different types:


```

1      struct Point { int x; int y; };
2      union U { int a; float b; };

```

Statements and Blocks

The statement rule serves as a grouper for all the different types of executable statements in C, including control flow structures such as loops (while, for), conditionals (if), and control statements (continue, break, goto, return). An expression followed by a semicolon is also considered a statement.

```

1 statement : while_stmt
2           | if_stmt
3           | for_stmt
4           | switch_stmt
5           | continue_stmt
6           | break_stmt
7           | goto_stmt
8           | return_stmt
9           | expression_stmt

```

Code blocks are represented by the block_lines and block_line rules, allowing the organization of multiple statements and declarations within curly braces (), essential for functions and control structures.

Example:

```

1 {
2     int i = 0;
3     while(i < 10) i++;
4 }

```

Expressions

The analysis of expressions follows a well-defined hierarchy to ensure the correct precedence of operators. This structure is essential for consistently evaluating complex expressions, starting with the simplest expressions and progressing to the more complex ones.

- `expression` → `assignment_expression` → `logical_exp` → `shift_exp` → `addsub_exp` → `muldiv_exp` → `unary_exp` → `complex_exp` → `simple_expression`

This cascade of rules allows the parser to recognize everything from basic arithmetic operations to compound assignments and complex logical operations, respecting the order of evaluation.

```

1 expression : assignment_expression
2             | expression COMMA assignment_expression
3 addsub_exp : muldiv_exp
4             | addsub_exp PLUS muldiv_exp
5             | addsub_exp MINUS muldiv_exp

```

Example:

```

1 a = b + 2 * (c - 1);

```

Compound Assignment and Unary Operators

The `assign_operator` rule covers all compound assignment operators available in C, which combine an arithmetic or bitwise operation with an assignment.

```

1 assign_operator : ASSIGN
2                 | ASSIGN_PLUS
3                 | ASSIGN_MINUS
4                 | ASSIGN_MULT
5                 | ASSIGN_DIV
6                 | ...

```

This allows concise assignment operations to be recognized:

```

1 x += 3;
2 y <<= 1;
3 *p = 10;

```

Unary operators, which operate on a single operand (such as increment, decrement, pointer dereference, and address-of), are handled by the `unary_exp` and `unary_operator` rules. These rules distinguish between prefix and postfix operators.

```

1 unary_exp : complex_exp
2            | PLUSPLUS unary_exp

```

```

3         | MINUSMINUS unary_exp
4         | ID PLUSPLUS
5         | ID MINUSMINUS
6         | ASTERISK unary_exp
7         | BIT_AND unary_exp

```

Loops: While, For, Do-While

Loop analysis is handled by the `while_stmt` and `for_stmt` rules, which support the various valid forms of these control flow structures, including the omission of parts in the `for` loop declaration.

```

1 while_stmt : WHILE OPEN_PAREN expression_stmt CLOSE_PAREN OPEN_CURLY
              block_lines CLOSE_CURLY
2           | ...
3 for_stmt  : FOR OPEN_PAREN for_init_opt SEMICOLON for_cond_opt
              SEMICOLON for_iter_opt CLOSE_PAREN for_body

```

Example:

```

1 for(int i=0; i<10; i++) { ... }
2 while(x < 100) x++;

```

Conditional Statements: IF/ELSE

The `if_stmt` rule covers all possible variants of the `if-else` conditional statement, including multiple `else if` clauses and the possibility of empty blocks or single statements without braces.

```

1 if_stmt : IF OPEN_PAREN expression_stmt CLOSE_PAREN OPEN_CURLY
           block_lines CLOSE_CURLY ELSE OPEN_CURLY block_lines CLOSE_CURLY
2         | IF OPEN_PAREN expression_stmt CLOSE_PAREN expression_stmt
3         | ...

```

Example:

```

1 if (a > 0) { foo(); }
2 else if (a == 0) { bar(); }
3 else { baz(); }

```

Switch/Case

Recognition of the switch statement, with multiple case labels and an optional default label, is performed by the switch_stmt, case_seq, and case_stmt rules.

```
1 switch_stmt : SWITCH OPEN_PAREN ID CLOSE_PAREN OPEN_CURLY case_seq
               CLOSE_CURLY
2 case_seq   : case_stmt
3             | case_seq case_stmt
4 case_stmt  : CASE expression_stmt TWO_DOTS block_lines break_stmt
5             | DEFAULT TWO_DOTS block_lines break_stmt
```

Example:

```
1 switch(sel) {
2     case 0: doA(); break;
3     default: doC(); break;
4 }
```

Specific Declarations: Struct, Union, Enum, Typedef

The rules for struct, union, enum, and typedef allow for the declaration, definition, and initialization of these data structures and custom types. These rules are essential for modeling complex data types in C.

```
1 declaration : STRUCT ID OPEN_CURLY declaration_seq CLOSE_CURLY
               SEMICOLON
2             | UNION ID OPEN_CURLY declaration_seq CLOSE_CURLY
               SEMICOLON
3             | ENUM ID OPEN_CURLY enumerator_list CLOSE_CURLY
               SEMICOLON
4             | TYPEDEF declaration_specifiers ID SEMICOLON
```

Example:

```
1 typedef unsigned char BYTE;
2 enum Colors { RED, GREEN = 2, BLUE };
3 struct X { int a; float b; };
```

Preprocessor Directives

The `define_directive` rule is responsible for analyzing the various preprocessor directives, such as `#define`, `#undef`, `#ifdef`, `#ifndef`, and `#pragma`. These directives are processed before compilation and affect how the code is compiled.

```
1 define_directive : PREPROCESS_DEFINE ID NUMBER
2                  | PREPROCESS_DEFINE ID STRING
3                  | PREPROCESS_UNDEF ID
4                  | PREPROCESS_IFDEF ID
5                  | PREPROCESS_IFNDEF ID
6                  | PREPROCESS_PRAGMA pragma_body
7                  | ...
```

Example:

```
1 #define SIZE 16
2 #undef X
3 #ifdef DEBUG
4 #endif
```

8051 Extensions

In addition to standard C, the grammar incorporates specific tokens for the 8051 microcontroller extensions. These tokens allow the recognition of memory modifiers and declarations for Special Function Registers (SFRs), which are vital for embedded programming.

```
1 %token SFR DATA IDATA BDATA XDATA PDATA CODE BIT REENTRANT USING AT
   INTERRUPT SBIT
```

These tokens allow support for 8051-specific declarations, such as:

```
1 __xdata int buffer[64];
2 SFR P0 = 0x80;
3 __interrupt void ISR(void) { ... }
```

Summary

The implemented grammar is robust and comprehensive, covering the main constructs of the C language along with the specific extensions for the 8051 microcontroller. It was designed to recognize multiple syntactic forms of each statement, declaration, block, function, operator, directive, and control command.

4.2.2 AST(Abstract Syntax Tree)

In the implementation phase, the AST is dynamically constructed during parsing using the Bison grammar rules. Each production rule is associated with semantic actions that instantiate and connect `TreeNode` structures according to the grammar's logical structure. These nodes are linked using child and sibling relationships, reflecting the nesting and sequence of constructs in the source program.

The resulting AST provides a modular and extensible framework for subsequent compiler phases, including semantic analysis, symbol table population, and code generation. This section presents the concrete implementation of the AST, including the `TreeNode` structure, node classification, helper functions, and examples of how the parser maps grammar rules to AST representations.

TreeNode Structure

The `TreeNode` structure is the fundamental building block of the Abstract Syntax Tree (AST) in this compiler. Each `TreeNode` instance represents a node in the tree and encapsulates both structural and semantic information relevant to the syntactic element it models.

Listing 4.16: `TreeNode` Struct

```
1 typedef struct treeNode{
2
3     struct treeNode * pchild;
4     struct treeNode * psibling;
5     int lineno;
6     NodeKind nodekind;
7     attr nodeAttr;
8
9 }TreeNode;
```

The fields of the `TreeNode` are described as follows:

- `pchild`: A pointer to the first child of the node. This models hierarchical nesting (e.g., a block contains declarations and statements).
- `psibling`: A pointer to the next sibling node at the same hierarchical level, allowing the tree to represent sequences such as parameter lists or block statements.
- `lineno`: The line number from the source code associated with the node, useful for debugging and error reporting.
- `nodekind`: An enumerated value (`NodeKind`) indicating the specific type of the node (e.g., `funcDefNK`, `opNK`, `idNK`).
- `nodeAttr`: A union-like structure (`attr`) that stores additional attributes such as identifier names, numeric values, or operator types, depending on the node kind.

This flexible and lightweight design allows the AST to represent a wide variety of language constructs efficiently, while also supporting traversal and manipulation during later compilation stages.

Node Typing and Kinds

The compiler uses several enumerated types to categorize and describe the different kinds of nodes that may exist in the Abstract Syntax Tree (AST). These enums allow the system to consistently identify the role, behavior, and data associated with each node, while keeping the tree structure extensible and modular.

Listing 4.17: Node Types and Kinds

```

1
2 typedef enum {funcDefNK, blockNK, funcCallNK, funcDeclNK, argsNK,
   parameterNK, declarNK,
3       whileNK, doNK, ifNK, thenNK, elseNK, switchNK, caseNK,
   continueNK, breakNK, gotoNK, returnNK,
4       opNK, pointerNK, idNK, numberNK, fnumberNK, stringNK,
   charValueNK, castNK, derefNK,
5       typeNK, forNK, structNK, unionNK, typedefNK, defineNK,
   errorNK, lineNK, pragmaNK, undefNK, ifdirNK, ifdefNK,
   ifndefNK, elsedefNK, elifNK, endifNK, qualifierNK,
   storageNK, enumNK, arrayNK, arrayRowNK, arrayAccessNK,
   indexNK, memberAccessNK, pointerAccessNK} NodeKind;

```

```

6
7 typedef enum {voidK, boolK, charK, shortK, intK, longK, floatK,
    doubleK, signedK, unsignedK} Declark;
8
9 typedef enum {constK, volatileK} QualifierK;
10
11 typedef enum {autoK, staticK, registerK, externK} StorageK;
12
13 typedef enum {assign_divK, assign_percentK, assign_andK, assign_orK,
    assign_xorK, assign_left_shiftK, assign_right_shiftK, assignK,
    assign_plusK, assign_minusK, assign_multK, equal_equalK,
14         plusK, minusK, asteriskK, right_slashK, percentK,
15         logic_notK, logic_andK, logic_orK, greater_thanK,
            greater_than_equalK, smaller_thanK,
            smaller_than_equalK, differentK,
16         bit_notK, bit_andK, bit_orK, bit_xorK, right_shiftK,
            left_shiftK,
17         arrowK, plusplusK, minusminusK, commaK, ternaryK, dotK,
            plusplusPosK, minusminusPosK} opK;
18
19 typedef union {double fval; int val; char * name; char cval;} attr;

```

NodeKind The NodeKind enumeration defines the general category or role of each AST node. Examples include:

- funcDefNK, funcCallNK, parameterNK: Represent elements related to function definitions and calls.
- ifNK, whileNK, forNK, returnNK: Represent control structures.
- declarNK, typeNK, storageNK, qualifierNK: Represent declarations, storage classes, and type qualifiers.
- idNK, numberNK, charValueNK, stringNK: Represent literals and identifiers.
- opNK, castNK, derefNK, arrayAccessNK, pointerAccessNK: Represent expressions and operators.

This wide variety of node kinds ensures that the AST can model the full range of language constructs, including expressions, declarations, and compound statements.

DeclarK The DeclarK enumeration defines primitive data types such as:

- `intK`, `charK`, `floatK`, `doubleK`, `boolK`, etc.

Each declaration node includes a DeclarK value indicating its base type.

QualifierK and StorageK The QualifierK and StorageK enumerations specify optional modifiers for declarations:

- **QualifierK:** Includes `constK`, `volatileK`.
- **StorageK:** Includes `staticK`, `externK`, `autoK`, and `registerK`.

These are represented as child nodes within declaration chains.

opK The opK enumeration represents all possible operators, including:

- Arithmetic: `plusK`, `minusK`, `assignK`, etc.
- Logical: `logic_andK`, `logic_orK`, `equal_equalK`, etc.
- Bitwise and compound assignment: `bit_andK`, `assign_plusK`, etc.
- Unary/postfix operators: `plusplusK`, `plusplusPosK`, etc.

Each operator node in the AST (opNK) uses a value from opK to specify the operation being performed.

attr union The attr union allows nodes to carry a single associated value, which could be:

- `val` — integer value (e.g., for number literals)
- `fval` — floating-point value
- `name` — string identifier (e.g., variable/function name)
- `cval` — character constant

Each `TreeNode` uses this attribute container to store its most relevant data, depending on the node kind.

This flexible typing system allows the compiler to uniformly manage different constructs while preserving semantic richness and extensibility across the AST.

Helper Functions

To facilitate the construction and management of the Abstract Syntax Tree, several helper functions are implemented. These functions encapsulate common operations such as creating new nodes, and linking nodes as children or siblings, thereby promoting code modularity, readability, and reuse. The helper functions ensure that the AST maintains a consistent structure, simplifies tree manipulation, and abstracts away low-level pointer management from the rest of the compiler.

createNode()

Listing 4.18: createNode() Function

```
1  TreeNode * createNode(NodeKind kind){
2      TreeNode * t = (TreeNode *) malloc(sizeof(TreeNode));
3      int i;
4      if (t==NULL)
5          printf("Out of memory error at line\n");
6      else {
7          t->nodekind = kind;
8          t->lineno = lineno;
9
10         t->pchild = NULL;
11         t->psibling = NULL;
12
13         t->nodeAttr.val = 0;
14         t->nodeAttr.fval = 0.0;
15         t->nodeAttr.name = NULL;
16     }
17
18     return t;
19 }
```

This function dynamically allocates a new AST node (TreeNode) and initializes its fields: it sets the node type (nodekind), line number (lineno), and defaults for child/sibling pointers (pchild, psibling). It also resets attributes (like val, fval, name) to zero/NULL. If memory allocation fails, it prints an error but still returns the (NULL) node.

addChild()

Listing 4.19: addChild() Function

```

1  TreeNode * addChild(TreeNode *pparent, TreeNode *pnchild) {
2
3      if (pparent==NULL || pnchild==NULL)
4          printf("Invalid parameters\n");
5
6      if (pparent->pchild==NULL) {
7          pparent->pchild = pnchild;
8      } else {
9          TreeNode * sibling = pparent->pchild;
10         while (sibling->psibling)
11             sibling = sibling->psibling;
12         sibling->psibling = pnchild;
13     }
14     return pparent;
15 }

```

This function links a child node (pnchild) to a parent node (pparent) in an AST. If the parent has no existing children, it directly attaches the child; otherwise, it traverses to the end of the sibling chain and appends the new child. Returns the parent node (or warns if parameters are invalid).

addSibling()

Listing 4.20: addSiblings() Function

```

1  TreeNode* addSibling(TreeNode* node, TreeNode* sibling) {
2      if (node == NULL) return sibling;
3      TreeNode* temp = node;
4      while (temp->psibling) temp = temp->psibling;
5      temp->psibling = sibling;
6      return node;
7  }

```

This function appends a sibling node to the end of a node's sibling chain. If the input node is NULL, it returns the sibling directly. Otherwise, it traverses to the last sibling in the list and links the new sibling there, maintaining the tree structure.

Tree Design

Function Definition

Listing 4.21: Function Code

```
1
2 declaration_specifiers ID OPEN_PAREN parameter_list CLOSE_PAREN
  OPEN_CURLY block_lines CLOSE_CURLY
3
4 {
5     TreeNode* funcNode = createNode(funcDefNK
6         ); // Nó da função
7
8     TreeNode* idNode = createNode(idNK);
9     printf("criado_idNK_em_linha_X\n");
10    // Nó do identificador
11    idNode->nodeAttr.name = strdup($2);
12    // Armazena o nome do ID
13
14    addChild(funcNode, $1); // Tipo de
15    retorno
16    addChild(funcNode, idNode); // Nome da
17    função como nó filho
18    addChild(funcNode, $4); // Parâmetros
19    addChild(funcNode, $7); // Corpo da
20    função
21
22    $$ = funcNode;
23
24    printf("Function_definition: Type: %s,
25        Name: %s\n", $1, $2);
26
27 }
```

```

funcDefNK
  Tipo: void
  idNK: func
  parameterNK
    Tipo: char
    idNK: a
  blockNK:
    DeclarNK
      Tipo: int
      idNK: b
    DeclarNK
      Tipo: double
      idNK: c
    DeclarNK
      Tipo: float
      idNK: d

```

Figure 18: Function Tree

The image displays the generated AST from the code, with the root node (funcDefNK) representing the function, including children for the return type, name (idNK), parameters, and body (blockNK). The hierarchical structure mirrors the source code organization, enabling semantic analysis

For

Listing 4.22: For Code

```

1 for_stmt: FOR OPEN_PAREN for_init_opt SEMICOLON for_cond_opt
  SEMICOLON for_iter_opt CLOSE_PAREN for_body
2 {
3     TreeNode* FORnode = createNode(forNK);
4     addChild(FORnode, $3 ? $3 : createNode(blockNK));
5     addChild(FORnode, $5 ? $5 : createNode(blockNK));
6     addChild(FORnode, $7 ? $7 : createNode(blockNK));
7     addChild(FORnode, $9);
8     $$ = FORnode;

```

```

9      printf("Parsed complete for statement\n");
10  }

```

```

For:
  opNK: =
    DeclarNK
      Tipo: int
      idNK: a
      numberNK: 5
  opNK: <
    idNK: a
    numberNK: 2
  opNK: ++
    idNK: i
  blockNK:
    opNK: =
      DeclarNK
        Tipo: int
        idNK: a
        numberNK: 9

```

Figure 19: For Tree

The image shows the AST structure of a for loop, with nodes for initialization (For_init), condition (for_condition), iteration (for_iteration), and body (blockNK). This hierarchical representation captures the loop's logic and flow, enabling compiler analysis and optimization.

If

Listing 4.23: If Code

```

1 IF OPEN_PAREN expression_stmt CLOSE_PAREN OPEN_CURLY block_lines
  CLOSE_CURLY ELSE OPEN_CURLY block_lines CLOSE_CURLY
2 {
3     TreeNode* ifnode = createNode(ifNK);
4

```

```

5      TreeNode * thenNode = createNode(thenNK);
6      addChild(ifnode, thenNode);
7      addChild(thenNode, $3);
8
9      addChild(ifnode, $6);
10
11     TreeNode* elseNode = createNode(elseNK);
12     addChild(elseNode, $10);
13     addChild(ifnode, elseNode);
14     $$ = ifnode;
15
16     printf("Parsed if else stmt\n");

```

```

If:
  Then:
    opNK: ==
    idNK: j
    numberNK: 20
  blockNK:
    opNK: =
    DeclarNK
      Tipo: int
      idNK: a
    opNK: +
    idNK: b
    numberNK: 1
  Else:
    blockNK:
      opNK: =
      idNK: c
      numberNK: 4

```

Figure 20: If Tree

The AST (Abstract Syntax Tree) of if-else has a root node `ifNK` controlling two main blocks: `thenNK` (for condition and main block) and `elseNK` (for the alternative block). The parser uses `addChild()` to connect everything - first the condition (`$3`), then the blocks (`$6` for `then`, `$10` for `else`).

Variable Initialization

Listing 4.24: If Code

```
1
2 declaration_specifiers ID ASSIGN logical_exp SEMICOLON
3     {
4         TreeNode* typeNode = $1;
5
6         TreeNode* idNode = createNode(idNK);
7         idNode->nodeAttr.name = strdup($2);
8
9         TreeNode* declarNode = createNode(declarNK);
10        addChild(declarNode, typeNode);
11        addChild(declarNode, idNode);
12
13        TreeNode* assignNode = createNode(opNK);
14        assignNode->nodeAttr.val = assignK;
15
16        addChild(assignNode, declarNode);
17        addSibling(declarNode, $4); // $4 = expressão
18
19        $$ = assignNode;
20
21        printf("Declared variable with init: %s = ... \n",
22               $2);
23    }
```

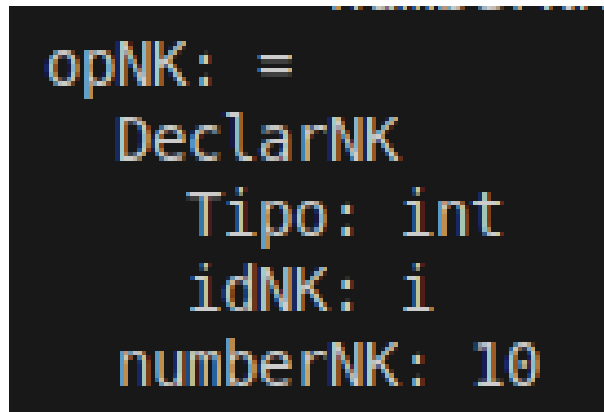



Figure 21: Variable Initialization Tree

The AST constructs variable initialization as: $\text{opNK}(\text{assignK}) \rightarrow \text{declaNK}(\text{type} + \text{idNK}) \parallel$ expression. Fig. 23 shows this structure.

Const Variable Declaration

Listing 4.25: If Code

```

1
2 declaration_specifiers : declaration_types
3 {
4     $$ = $1;
5 }
6 | storage_modifier declaration_types
7 {
8     addSibling($1, $2);
9     $$ = $1;
10 }
11 | type_qualifier declaration_types
12 {
13     addSibling($1, $2);
14     $$ = $1;
15 }
16 | storage_modifier type_qualifier declaration_types
17 {
18     addSibling($2, $3);
19     addSibling($1, $2);
20

```

```

21     $$ = $1;
22 }
23 ;

```

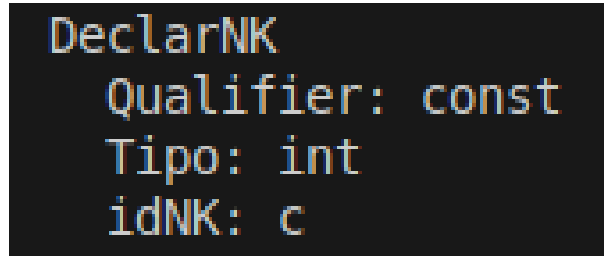


Figure 22: Const Variable Tree

The AST rules support any storage/type qualifier combo (e.g., static const). The image shows just the const case, but the addSibling() logic works universally.

Function Call

Listing 4.26: If Code

```

1
2 ID OPEN_PAREN value_list CLOSE_PAREN
3     {
4         TreeNode* callNode = createNode(funcCallNK);
5         TreeNode* idNode = createNode(idNK);
6         idNode->nodeAttr.name = strdup($1);
7
8         TreeNode* argsNode = createNode(argsNK);
9         addChild(argsNode, $3);
10
11         addChild(callNode, idNode);
12         addChild(callNode, argsNode);
13
14         $$ = callNode;
15         printf("SimpleExp: function call\n");
16     }

```

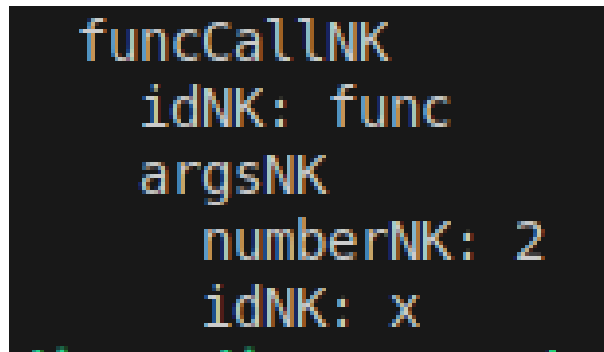


Figure 23: Function Call Tree

AST for function calls: `funcCallNK` $\rightarrow$ [`idNK` (name), `argsNK` (arguments)]. The image shows this hierarchy

Summary

The Abstract Syntax Tree structures we've examined reveal how a compiler systematically breaks down code into hierarchical relationships. For function calls like `func()`, the AST creates a clear parent-child structure with the function name as one branch and its arguments as siblings under an `argsNK` node. Variable declarations follow a similar logic but with added flexibility - whether we're dealing with simple types like `int` or complex modifiers like `static const`, the AST chains these elements through sibling nodes while preserving their semantic order.

Control flow statements demonstrate particularly elegant tree structures. An `if-else` block becomes a clean decision tree in the AST, with condition and branch blocks neatly organized under their respective nodes. For loops get decomposed into their four fundamental components (initialization, condition, update, body), allowing the compiler to analyze and optimize each piece independently.

What makes this AST design practical is its consistent patterns. The same `addChild()` and `addSibling()` operations handle everything from simple variable assignments to complex function calls. This uniformity matters because it lets the compiler apply the same analysis and optimization passes across different language constructs. The tree structure isn't just storing syntax - it's capturing the actual logic flow of the program in a way the compiler can work with efficiently.

The accompanying diagrams don't just illustrate these concepts - they show how the AST maintains a direct visual correspondence with the source code. When you see a function call node branching into its arguments, or a `const` modifier linking to its type, you're seeing the compiler's understanding of the code take shape. This is where syntax becomes semantics, and where the compiler begins the real work of turning human-written code into optimized machine operations

4.3 Semantic Analysis

The Semantic Analysis phase is responsible for ensuring that the syntactically correct program also adheres to the semantic rules of the language. While parsing verifies structure, semantic analysis verifies meaning. This includes tasks such as type checking, scope resolution, validation of declarations and usages, and ensuring consistency in assignments and function calls.

In this compiler, semantic analysis is supported by a symbol table that tracks variable and function declarations, their types, scopes, and attributes. Additionally, mechanisms for type inference and type compatibility checking were implemented to handle implicit conversions and enforce type safety. This phase ensures that the constructed Abstract Syntax Tree (AST) reflects a semantically valid program before proceeding to code generation.

4.3.1 Symbol Table

Symbol Table Data Structures

The symbol table infrastructure is implemented using three main C data structures: `LineList`, `BucketList`, and `SymbolTable`. These collectively allow the compiler to store and query all declared identifiers in a way that supports scope nesting, efficient lookup, and flexible semantic tracking.

Line List

```
1 typedef struct LineListRec {
2     int lineno;
3     struct LineListRec *next;
4 } *LineList;
```

The `LineList` is a linked list of line numbers representing the occurrences of a particular symbol in the source code. Each `BucketList` node contains a `LineList` entry to track where the symbol was referenced. This is useful for later reporting of semantic warnings, such as unused variables or repeated references.

Bucket List

```
1 typedef struct BucketListRec {
2     char *name;
```

```

3      NodeKind kind;
4      Declark type;
5      StorageK storage;
6      QualifierK qualifier;
7      char isDefined;
8      char isPointer;
9      char isUnsigned;
10     char isArray;
11     int memloc;
12     LineList lines;
13     struct BucketListRec *next;
14 } *BucketList;

```

The BucketList structure is bucket in a symbol table, which stores information about a single symbol (identifier). The main fields are as follows:

- name – the symbol's identifier name.
- kind – the type of syntactic node (e.g., variable, function, etc.).
- type – the semantic type of the symbol (e.g., intK, floatK, etc.).
- storage – specifies the storage class (e.g., static, extern).
- qualifier – qualifiers such as const.
- isDefined – flag indicating whether the symbol was defined (1) or only declared (0).
- isPointer, isUnsigned, isArray – flags for language features that modify the base type.
- memloc – memory location (used during code generation).
- lines – a linked list of line numbers where the symbol is used.
- next – points to the next symbol in the hash bucket (chaining for collision resolution).

This structure enables full semantic tracking of each identifier and supports language features such as pointers, arrays, type qualifiers, and function signatures.

Symbol Table Structure

```
1 typedef struct SymbolTable {
2     BucketList* hashTable;
3     long capacity;
4     long nBucketsUsed;
5     struct SymbolTable *scope_parent;
6     struct SymbolTable *scope_child;
7     struct SymbolTable *scope_sibling;
8     char scopeLevel;
9 } SymbolTable;
```

The SymbolTable structure represents a scope in the program. Each table contains a dynamically sized hash table of BucketList entries and pointers to its parent, first child, and sibling scopes. This makes it possible to navigate and traverse the scope hierarchy, respecting C's lexical scoping rules.

- hashTable – array of buckets used for fast symbol lookup.
- capacity – current size of the hash table (used for resizing).
- nBucketsUsed – current number of filled buckets (used to decide when to expand).
- scope_parent, scope_child, scope_sibling – used to represent the scope tree (hierarchical structure).
- scopeLevel – indicates the depth level of the scope, starting from 0 for global.

The table hierarchy and dynamic hash allocation ensure scalable symbol tracking and quick resolution across nested contexts.

Hashing Function

```
1 int hash_with_capacity(char* key, int capacity) {
2     unsigned int hashval = 0;
3     while (*key != '\0')
4         hashval = (hashval << 4) + *(key++);
5     return hashval % capacity;
6 }
```

The function `hash_with_capacity` implements a left-shift-add hashing algorithm. For each character in the input key, it shifts the current hash value left by 4 bits and adds the ASCII value of the character. After processing all characters, it reduces the result modulo the current hash table capacity, ensuring that the output index fits within the bucket array.

This approach balances simplicity and uniform distribution, providing efficient bucket assignment while supporting resizing. The hash function is used both during insertion and lookup to locate the correct bucket.

Constants and Initial Configuration

Two key constants control the symbol table's configuration:

```
1 #define INITIAL_CAPACITY 79 // initial number of buckets (prime
   number)
2 #define MAX_BUCKET_CAPACITY 131072 // maximum hash table capacity
   (217)
```

These constants define the default and maximum size of the hash table. The initial capacity of 79 is chosen to reduce collision risk, while the maximum capacity sets an upper bound to avoid unbounded memory usage during resizes.

Symbol Table Creation

The function `createTable` is used to initialize a new symbol table (representing a new scope):

```
1 SymbolTable* createTable(SymbolTable* parent, char level) {
2     SymbolTable *t = malloc(sizeof(SymbolTable));
3     t->capacity = INITIAL_CAPACITY;
4     t->nBucketsUsed = 0;
5     t->hashTable = calloc(t->capacity, sizeof(BucketList));
6     t->scope_parent = parent;
7     t->scope_child = NULL;
8     t->scope_sibling = NULL;
9     t->scopeLevel = level;
10    if (parent != NULL) {
11        if (parent->scope_child == NULL) {
12            parent->scope_child = t;
```

```

13     } else {
14         SymbolTable *s = parent->scope_child;
15         while (s->scope_sibling != NULL) {
16             s = s->scope_sibling;
17         }
18         s->scope_sibling = t;
19     }
20 }
21
22 return t;
23 }

```

The function first allocates memory for the SymbolTable structure. It initializes the internal hashTable to an array of empty buckets with an initial capacity defined by INITIAL_CAPACITY (a prime number like 79). The field nBucketsUsed is set to 0 to track how many buckets contain at least one entry.

Parent-child and sibling relationships are also set at this point. If the scope has a parent (e.g., this is a nested block or function scope), the new table is attached to the parent as a child scope. If the parent already has a child scope, the function traverses the sibling list until the last one and attaches the new scope there. This forms a tree structure of nested scopes.

Symbol Insertion and Table Resizing

The st_insert function inserts a new identifier or updates an existing one. If the table is too full, it resizes itself and rehashes all existing symbols.

```

1 void st_insert(SymbolTable* table, char *name, NodeKind kind, DeclaRK
   type, StorageK storage, QualifierK qualifier, char isDefined,
2 int lineno, int memloc) {

```

The function first checks whether the table needs resizing by comparing nBucketsUsed to half the capacity. This strategy ensures that the load factor stays below 0.5, reducing collisions. If resizing is necessary, a new table is created, and all existing entries are reinserted into the new table using the updated hash function.

```

1     if (table->nBucketsUsed * 2 >= table->capacity) {
2         if (table->capacity >= MAX_BUCKET_CAPACITY) {
3             fprintf(stderr, "Erro: capacidade máxima atingida.\n");

```



```

4     exit(EXIT_FAILURE);
5 }
6
7     int oldCapacity = table->capacity;
8     BucketList* oldTable = table->hashTable;
9
10    table->capacity *= 2;
11    table->hashTable = calloc(table->capacity, sizeof(BucketList)
12                               );
13    table->nBucketsUsed = 0;
14
15    for (int i = 0; i < oldCapacity; ++i) {
16        BucketList l = oldTable[i];
17        while (l != NULL) {
18            BucketList next = l->next;
19            int h = hash_with_capacity(l->name, table->capacity);
20            if (table->hashTable[h] == NULL)
21                table->nBucketsUsed++;
22            l->next = table->hashTable[h];
23            table->hashTable[h] = l;
24            l = next;
25        }
26    }
27
28    free(oldTable);
29 }

```

Afterward, the function computes the hash index for the name. If the symbol already exists (determined by linear search in the collision list), the line number is appended to its LineList. Otherwise, a new entry is created and prepended to the bucket's linked list.

Symbol Lookup

The function `st_lookup` searches for a symbol starting from the current scope and moving up through parent scopes until the symbol is found or the root is reached:

```

1 BucketList st_lookup(SymbolTable* table, const char *name) {

```

```

2 SymbolTable *current = table;
3 while (current != NULL) {
4     int h = hash_with_capacity((char*)name, current->capacity);
5     BucketList l = current->hashTable[h];
6     while (l != NULL) {
7         if (strcmp(name, l->name) == 0) {
8             return l;
9         }
10        l = l->next;
11    }
12    current = current->scope_parent;
13 }
14
15 return NULL;
16 }

```

This implementation ensures that the nearest (most local) declaration of a variable is always returned, honoring lexical scope rules. It uses ‘hash_with_capacity’ at each level of the scope tree, recalculating the bucket index for the current table’s capacity. It returns an entry in the symbol table (if existing), which is a BucketList struct.

Local Symbol Lookup

In contrast, st_lookupLocal performs a lookup exclusively within the current scope:

```

1 BucketList st_lookupLocal(SymbolTable* table, const char name) {
2     int h = hash_with_capacity((char)name, table->capacity);
3     BucketList l = table->hashTable[h];
4     while (l != NULL) {
5         if (strcmp(name, l->name) == 0) return l;
6         l = l->next;
7     }
8     return NULL;
9 }

```

This is particularly useful during declaration, where we must check if a variable has already been

declared in the current block. It uses the same hash and traversal logic but only for the current scope table.

Symbol Table Construction and AST Traversal

The construction of symbol tables in the compiler is performed by traversing the AST and inserting symbols encountered at each node into the appropriate symbol table. This is accomplished through a combination of recursive tree traversal and scope management logic.

Top-Level Symbol Table Construction Function

```
1 void buildSymtab(TreeNode* syntaxTree) {  
2     globalTable = createTable(NULL, 0);  
3     currentTable = globalTable;  
4     traverse(syntaxTree, insertNode, nullProc);  
5 }
```

The buildSymtab function initializes the global scope by calling createTable with a NULL parent. It then invokes traverse, a recursive function designed to walk the AST. The insertNode function is passed as the pre-order function to be executed at each node, which handles the logic of symbol insertion. nullProc is a placeholder post-order function that does nothing. globalTable and currentTable are two global pointers that are used to manage the symbol table state during the construction of the symbol tables.

Recursive Tree Traversal and Scope Management

The traverse function is the core of the symbol table construction mechanism. It accepts two function pointers: one for pre-order processing (executed before children are traversed), and one for post-order processing (executed after).

The function is defined as:

```
1 void traverse(TreeNode* t,  
2             void (*preProc)(TreeNode*),  
3             void (*postProc)(TreeNode*));
```

The depth-first strategy utilized during traversal involves the preProc() function being invoked at each node t, followed by a recursive traversal of all children, and finally the postProc() function. Symbol insertion is executed using pre-order.

Each node is annotated with its associated scope by assigning:

```
1 if (preProc == insertNode)
2     t->scope = currentTable;
```

If the scope is invalid, such as being NULL or lacking a valid hash table, a diagnostic warning is issued, which aids in the early detection of uninitialized or misconfigured scopes.

New scopes are introduced dynamically for specific node kinds. These include:

- Function definitions: `funcDefNK`
- Compound blocks: `newblockNK`
- Control structures: `ifNK`, `elseNK`, `whileNK`, `forNK`, `switchNK`

In such cases, a new symbol table is allocated via `createTable()`, linked to the current one, and assigned to `currentTable`:

```
1 SymbolTable* newScope = createTable(currentTable,
2                                     currentTable->scopeLevel + 1);
3 currentTable = newScope;
```

A special case is handled for `elseNK` blocks, which inherit the scope level of the corresponding `ifNK` but use the grandparent scope as their parent. This design ensures that both branches of the conditional have access to the same variables without clashing scope identifiers.

Once the subtree of a scoped node is fully traversed, the previous scope is restored:

```
1 if (createdScope) {
2     currentTable = oldTable;
3 }
```

This disciplined scope management guarantees consistent nesting, prevents leakage of inner declarations, and ensures proper resolution of identifiers across different blocks.

Enforcement of Semantic Rules

The `insertNode()` function is invoked in pre-order during the AST traversal (via the `traverse()` function) and is responsible for inserting identifiers (variables, functions, types, macros) into the appropriate symbol table for the current scope. Additionally, it enforces some semantic rules required by the C language specification, such as preventing duplicate declarations in the same scope and validating function definitions against previous declarations.

The function uses a `switch` statement on the `nodekind` of each AST node to determine the appropriate handling strategy. Below is an overview of how the major node types are processed:

Function Declarations and Definitions When encountering `funcDeclNK` or `funcDefNK`, the function parses the child nodes to extract modifiers (e.g., `unsigned`, `static`, `pointer`), the return type, and the identifier name. If the function is a definition, it ensures that the function has not already been defined and that the types match any prior declarations.

```
1 if (entrada && entrada->isDefined) {  
2     printf("Erro: função '%s' já definida.\n", nome);  
3 }
```

If the function was declared but not defined, it verifies that the return type and parameter list match.

Variable and Parameter Declarations Declarations (`declarNK`, `parameterNK`) are similarly parsed to extract type information, pointers, arrays, and qualifiers. Each identifier is then inserted into the current scope's symbol table, unless a duplicate already exists.

```
1 if (st_lookupLocal(currentTable, nome)) {  
2     printf("Erro: variável '%s' já declarada neste scope.\n", nome);  
3 } else {  
4     st_insert(currentTable, nome, kind, tipo, storage, qualifier, 1,  
5         t->lineno, location++);  
6 }
```

The function also detects enumeration declarations and inserts enum constants as integer literals.

Typedefs The `typedefNK` node is processed by storing the new type alias and associated modifiers in the symbol table.

ID Usage Validation When an `idNK` node is encountered (e.g., usage of a variable), the function performs a lookup to ensure the symbol has been previously declared. If the symbol does not exist in any accessible scope, an error is raised.

```
1 BucketList entrada = st_lookup(currentTable, t->nodeAttr.name);  
2 if (!entrada) {  
3     printf("Erro: identificador '%s' usado mas não declarado previamente  
4         .\n", t->nodeAttr.name);
```

Error Handling Throughout the function, various semantic checks are performed and errors are printed immediately when detected. For example, attempting to declare the same parameter name twice in a function will result in an error.

Structure and Extensibility The function is modularly organized using a switch-case style, making it easy to extend for other node types (e.g., struct declarations, inline functions, global constants) by adding new cases. It leverages helper functions like `st_lookupLocal()`, `st_insert()`, and `parametrosIguais()` to enforce consistent behavior across declarations.

4.3.2 Type Checking

Structure of the `checkNode` Function

The `checkNode` function implements a recursive traversal of the Abstract Syntax Tree (AST). At each node, it outputs debug messages and invokes specific functions depending on the node type (`nodekind`), enabling modularization of the various semantic checks.

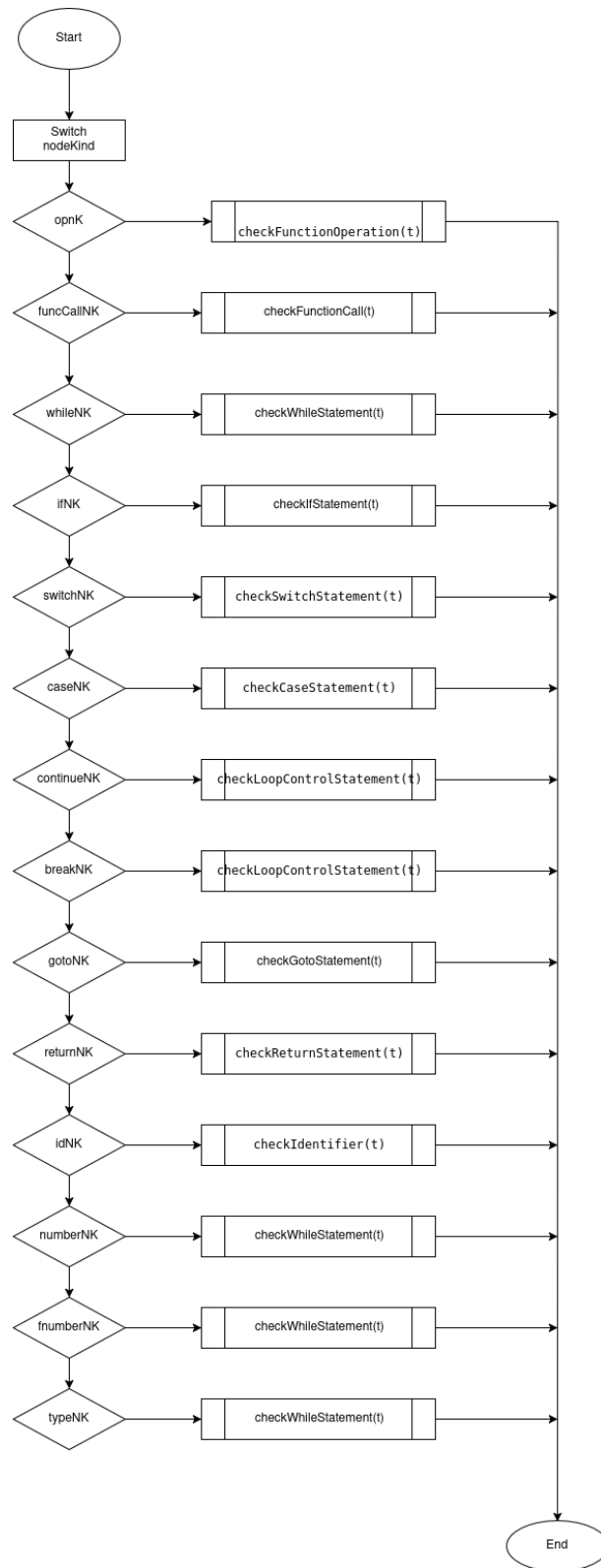


Figure 24: Flowchart for the checkNode function.

Example: checkFunctionOperation

The function `checkFunctionOperation` is responsible for performing semantic validation of operations within the syntax tree. It begins by detecting variable initialization, identifying declaration nodes (`declarNK`) on the left side of the operation. The function then extracts the associated type (`typeNK`) and assigns it to the corresponding identifier (`idNK`). For identifiers without an explicit type, the function infers their types based on the surrounding context. Additionally, numeric literals such as integers (`numberNK`) and floating-point numbers (`fnumberNK`) are assigned specific types.

The function proceeds to verify type compatibility. Operations involving identical types are considered valid, and certain type combinations are allowed—such as integer literals combined with variables and the promotion of float types to double. Assignments between compatible integer types are also accepted. If any type incompatibility is detected, the function calls `typeError` with a descriptive message to facilitate debugging.

```
1 // Debug messages
2 [DEBUG] Verificar operação '+'
3 [OK] Operação válida: tipos iguais => intK
4
5 // Error message
6 [TypeError] Linha 0: Tipos incompatíveis nos operandos da operação
```

Control Validation: checkIfStatement, checkWhileStatement, and checkSwitchStatement

The functions responsible for validating control structures ensure the semantic correctness of conditional and looping statements in the program.

For `if` and `while` statements, the condition must be of type `intK`, representing boolean values (true or false). If this is not the case, the validation fails, indicating a type mismatch for the condition.

In the case of `switch`, the function uses an auxiliary variable `insideSwitch` to track whether the current context permits the use of the `break` statement, preventing its misuse outside valid scopes. Furthermore, the expression that defines the `switch` must be of integer type (`intK`). The presence of at least one case and a `default` clause is required to ensure comprehensive handling of all possibilities.

Each case must contain an integer expression whose value is compatible with the type of the `switch`, ensuring semantic validity in matching and selecting the correct block.

Error Detection

Operations

Common errors include:

- Use of operators with incompatible types.
- Invalid assignments, for example, between `voidK` and `intK`.
- Use of literals with operators that do not support their types.

Example error message:

```
1 Different types in operation: voidK vs intK
```

Variables

```
1 // Error messages
2 [TypeError] Variable already declared
3 throwError(t, "Identifier_not_declared");
```

Control Flow

```
1 // Error messages
2 throwError(..., "Condition_must_be_boolean");
3 throwError(..., "break_outside_of_loop_or_switch");
```

Return Statements

```
1 // Error messages
2 Error: empty return in function with return type 'int'.
3 Error: incompatible return type.
```

Switch / Case

```
1 // Error messages
2 Error: missing case or default clause.
3 Error: incompatible types between switch expression and case labels.
```

Variáveis globais

```
1 #include "typecheck.h"
2
3 static int location = 0; // Para alocação de memória
4 static int insideSwitch = 0; // Para controle de escopo de switch
```

isNumeric

The function `isNumeric` checks if the given type is numeric (int, float or double):

```
1 int isNumeric(DeclarK type) {
2     return type == intK || type == floatK || type == doubleK;
3 }
```

It returns 1 (true) if the type is one of the numeric types, otherwise it returns 0 (false).

isComparable

The function `isComparable` checks whether two given types can be compared with each other:

```
1 int isComparable(DeclarK t1, DeclarK t2) {
2     if (isNumeric(t1) && isNumeric(t2)) return 1;
3     //if (isBoolean(t1) && isBoolean(t2)) return 1;
4     if (t1 == t2) return 1;
5     return 0;
6 }
```

It returns 1 if both types are numeric, or if they are equal, indicating that the comparison between them is valid. Otherwise, it returns 0.

promoteNumericType

The function `promoteNumericType` performs numeric type promotion by choosing the most "comprehensive" type between two given numeric types:

```
1 DeclarK promoteNumericType(DeclarK t1, DeclarK t2) {
2     if (t1 == doubleK || t2 == doubleK) return doubleK;
3     if (t1 == floatK || t2 == floatK) return floatK;
```

```

4 return intK;
5 }

```

If any of the types are doubleK, the result is doubleK. Otherwise, if any are floatK, return floatK. If neither is float or double, returns intK by default.

checkFunctionOperation

The function checkFunctionOperation is responsible for checking the type validity of expressions in the syntax tree. It performs **type inference** by analyzing the operands of an operation and deducing the resulting type based on the operator semantics and operand types.

```

1 void checkFunctionOperation(TreeNode* t) {
2     if (t == NULL) return;
3     TreeNode* left = t->pchild;
4     TreeNode* right = left ? left->psibling : NULL;
5
6     // Extract types from declarations or symbol table
7     ...
8
9     // Infer type of float literals
10    if (left->nodekind == fnumberNK) left->type = floatK;
11    if (right && right->nodekind == fnumberNK) right->type = floatK;
12
13    // Unary operations: result type = operand type
14    if (right == NULL) {
15        t->type = left->type;
16        return;
17    }
18
19    // Handle common numeric promotions
20    if (left->type == right->type) {
21        t->type = left->type;
22        return;
23    }
24

```

```

25 // Mixed int/float/double combinations
26 ...
27
28 // Final fallback: type error
29 throwError(t, "Incompatible operand types");
30 }

```

This function implements several key mechanisms:

- **Type inference:** When operand types are missing (e.g., identifiers), they are looked up in the symbol table.
- **Unary operations:** Propagate the operand type directly to the parent.
- **Literal handling:** Integer and floating-point literals are assigned their expected types (intK, floatK).
- **Type promotion:** Mixed numeric types are accepted when consistent (e.g., float + double yields double).
- **Special cases:** Integer literals can be combined with variables (e.g., char + 5), with the variable's type determining the result.
- **Type errors:** If no rule applies, an error is raised to flag incompatible types in the operation.

Debug messages are printed throughout the function to assist in development, indicating which rules were applied and where errors occurred. This function plays a central role in enforcing type safety during semantic analysis.

checkFunctionDefinition

The function `checkFunctionDefinition` verifies the correctness of a function definition:

```

1 static void checkFunctionDefinition(TreeNode* t) {
2     TreeNode* returnType = t->pchild;
3     TreeNode* id = returnType ? returnType->psibling : NULL;
4     TreeNode* paramList = id ? id->psibling : NULL;
5
6     if (returnType == NULL || returnType->nodekind != typeNK) {

```

```

7      typeError(t, "Function_definition_missing_return_type");
8      return;
9  }

10

11  if (id == NULL || id->nodekind != idNK) {
12      typeError(t, "Function_definition_missing_name");
13      return;
14  }

15

16  // Check if there's already a declaration/definition
17  BucketList entry = st_lookupLocal(globalTable, id->nodeAttr.name)
18      ;
19  if (entry != NULL) {
20      // Check if return types match
21      if (entry->type != returnType->nodeAttr.val) {
22          typeError(t, "Return_type_of_definition_differs_from_
23              declaration");
24      }
25      // Check if parameters match
26      if (!parametersMatch(entry->params, paramList, id->nodeAttr.
27          name)) {
28          typeError(t, "Parameters_of_definition_differ_from_
29              declaration");
30      }
31      entry->isDefined = 1;
32  } else {
33      // Insert new function in the symbol table
34      st_insert(globalTable, id->nodeAttr.name, funcDefNK,
35          returnType->nodeAttr.val, SC_NONE, NONE_QK, 1, t->lineno,
36          location++);
37  }
38  }

```

This function ensures the function has a return type and name, checks for prior declarations, verifies consistency of return types and parameters, and updates or inserts the function symbol accordingly.

checkFunctionCall

The function `checkFunctionCall` performs semantic validation of function call expressions. This includes checking for declaration, ensuring that the correct number and types of arguments are passed, and issuing appropriate warnings or errors.

```
1 void checkFunctionCall(TreeNode* t) {
2     if (!t || !t->pchild) return;
3
4     TreeNode* id = t->pchild;
5     TreeNode* args = id->psibling;
6
7     // Ensure identifier is valid and declared
8     ...
9
10    // Attempt to retrieve function definition or declaration
11    TreeNode* def = findNode(savedTree, funcName, funcDefNK);
12    if (!def) def = findNode(savedTree, funcName, funcDeclNK);
13
14    // Extract parameter declarations
15    TreeNode* paramDecl = ...;
16
17    // Match each argument with corresponding parameter
18    while (pDecl && pArg) {
19        Declark expectedType = ...;
20        Declark actualType = pArg->type;
21
22        if (expectedType != actualType) {
23            // Issue warning for type mismatch
24            ...
25        }
26
27        pDecl = pDecl->psibling;
28        pArg = pArg->psibling;
29    }
30}
```

```

31     // Check for argument count mismatch
32     ...
33 }

```

The semantic checks performed by this function include:

- **Identifier validation:** Ensures that the function being called is a valid identifier.
- **Declaration check:** Uses the symbol table to confirm that the function was declared. If not, an error is raised.
- **Definition lookup:** Searches the AST to locate the function's definition or declaration, enabling parameter verification.
- **Parameter extraction:** Retrieves the list of expected parameters from the function declaration or definition.
- **Type matching:** Compares the type of each actual argument to the corresponding expected parameter type. Type mismatches generate warnings.
- **Argument count check:** Verifies that the number of arguments passed matches the number of parameters. Discrepancies result in errors.

This validation ensures that function calls respect their interface contracts, reinforcing both type safety and arity correctness. It is also tolerant to minor inconsistencies (e.g., missing function definition) by issuing warnings instead of halting compilation, which supports better error recovery and analysis.

checkParameter

The function `checkParameter` validates a single function parameter declaration:

```

1 static void checkParameter(TreeNode* t) {
2     // Check if the parameter has a valid type
3     TreeNode* typeNode = t->pchild;
4     if (typeNode == NULL || typeNode->nodekind != typeNK) {
5         typeError(t, "Parameter missing type");
6         return;
7     }
8 }

```

```

9      // Check if the parameter has a valid name
10     TreeNode* id = typeNode->psibling;
11     if (id == NULL || id->nodekind != idNK) {
12         typeError(t, "Parameter missing name");
13         return;
14     }
15
16     // Check if the parameter was already declared in this scope
17     if (st_lookupLocal(currentTable, id->nodeAttr.name)) {
18         typeError(t, "Parameter already declared");
19         return;
20     }
21
22     // Insert parameter into the symbol table
23     id->type = typeNode->nodeAttr.val;
24     st_insert(currentTable, id->nodeAttr.name, t->nodekind, typeNode
25               ->nodeAttr.val, SC_AUTO, NONE_QK, 1, t->lineno, location++);
26     // Update the parameter node type
27 }

```

This function ensures that each parameter has a type and a name, that it is not redeclared in the current scope, and inserts it into the symbol table.

checkDeclaration

The function checkDeclaration verifies a variable declaration:

```

1 static void checkDeclaration(TreeNode* t) {
2     // Check if the declaration has a valid type
3     TreeNode* typeNode = t->pchild;
4     TreeNode* id = typeNode ? typeNode->psibling : NULL;
5     if (typeNode == NULL || typeNode->nodekind != typeNK) {
6         typeError(t, "Declaration missing type");
7         return;
8     }
9 }

```



```

10  if (id == NULL || id->nodekind != idNK) {
11      typeError(t, "Declaration_missing_name");
12      return;
13  }
14
15  // Check if the variable is already declared
16  if (st_lookupLocal(currentTable, id->nodeAttr.name)) {
17      typeError(t, "Variable_already_declared");
18      return;
19  }
20
21  id->type = typeNode->type;
22  //st_insert(currentTable, id->nodeAttr.name, declarNK, typeNode->
23      nodeAttr.val, SC_AUTO, NONE_QK, 1, t->lineno, location++);
24  st_lookup(currentTable, id->nodeAttr.name)->isDefined = 1; //
    Mark as defined
25  }

```

checkWhileStatement

The function checkWhileStatement validates a while loop statement:

```

1  static void checkWhileStatement(TreeNode* t) {
2      // Check if the while condition is valid
3      TreeNode* cond = t->pchild;
4      TreeNode* body = cond ? cond->psibling : NULL;
5
6      if (cond == NULL && cond->nodekind != opNK && cond->nodekind !=
7          idNK && cond->nodekind != numberNK) {
8          typeError(t, "Invalid_while_condition");
9          return;
10     }
11     traverse(cond, nullProc, checkNode);
12     if (cond->type != intK) {
13         typeError(t, "While_condition_must_be_boolean(int)");
14     }
15 }

```

```

13     }
14
15     // Check if the body of the while is valid (should be blockNK)
16     if (body == NULL || body->nodekind != blockNK) {
17         typeError(t, "Invalid_while_body");
18         return;
19     }
20
21     printf("[DEBUG]_Checking_while_body\n");
22     traverse(body, nullProc, checkNode);
23 }

```

checkIfStatement

The function checkIfStatement validates an if statement:

```

1 static void checkIfStatement(TreeNode* t) {
2     // Check if the if condition is valid
3     TreeNode* cond = t->pchild;
4     TreeNode* thenPart = cond ? cond->psibling : NULL;
5     TreeNode* elsePart = thenPart ? thenPart->psibling : NULL;
6
7     if (cond) {
8         traverse(cond, nullProc, checkNode);
9         if (cond->type != intK) {
10             typeError(t, "If_condition_must_be_boolean");
11         }
12     } else {
13         typeError(t, "Invalid_if_condition");
14     }
15
16     if (thenPart) {
17         traverse(thenPart, nullProc, checkNode);
18     } else {
19         typeError(t, "Invalid_'then'_part_of_if");

```

```

20     }
21
22     // else is optional: only check if present
23     if (elsePart) {
24         traverse(elsePart, nullProc, checkNode);
25     }
26 }

```

checkReturnStatement

The function checkReturnStatement validates a return statement:

```

1 static void checkReturnStatement(TreeNode* t) {
2     if (!currentFunction) {
3         printf("Error at line %d: return statement outside of any
4             function.\n", t->lineno);
5         return;
6     }
7
8     // Case of return without expression
9     if (t->pchild == NULL) {
10        if (currentFunction->type != voidK) {
11            printf("Error: empty return in function (%s) with return
12                type '%s'.\n",
13                currentFunction->name, typeKindToString(
14                    currentFunction->type));
15        }
16        return;
17    }
18
19    // Check the type of the returned value
20    DeclarkK exprType = t->pchild->type;
21
22    // If type is undefined (e.g. literal), try to determine
23    if (exprType == NONE_DK) {

```

```

21     if (t->pchild->nodekind == numberNK) {
22         exprType = intK;
23     } else if (t->pchild->nodekind == fnumberNK) {
24         exprType = floatK;
25     } else if (t->pchild->nodekind == idNK) {
26         BucketList sym = st_lookup(t->scope, t->pchild->nodeAttr.
                name);
27         if (sym) exprType = sym->type;
28     } else if (t->pchild->nodekind == opNK) {
29         exprType = t->pchild->type; // Should have been set by
                checkFunctionOperation
30     }
31 }
32
33 if (exprType != currentFunction->type) {
34     printf("Error: incompatible return type (%s). Expected '%s',
            got '%s'.\n",
35           currentFunction->name,
36           typeKindToString(currentFunction->type),
37           typeKindToString(exprType));
38 }
39 }

```

checkSwitchStatement

The function `checkSwitchStatement` performs semantic analysis of switch statements, validating the type of the controlling expression and the correctness of its associated case and default clauses.

```

1 static void checkSwitchStatement(TreeNode* t) {
2     TreeNode* expr = t->pchild;
3     TreeNode* caseNode = expr ? expr->psibling : NULL;
4
5     // Validate switch expression
6     traverse(expr, nullProc, checkNode);
7     if (!isInteger(expr->type)) {

```

```

8      throwError(expr, "Switch expression must be an integer");
9  }
10
11  // Traverse case/default nodes
12  while (caseNode) {
13      if (caseNode->nodekind == caseNK) {
14          // Check case expression and body
15          ...
16      } else if (caseNode->nodekind == defaultNK) {
17          // Check default body
18          ...
19      }
20      caseNode = caseNode->psibling;
21  }
22
23  // Enforce rules on number of cases/defaults
24  ...
25  }

```

This function enforces the following rules during semantic analysis:

- **Switch expression must be an integer:** The controlling expression is checked for integer type compatibility using the `isInteger` predicate. Non-integer expressions are flagged as errors.
- **Case clauses:** Each case clause must have:
 - An expression of integer type matching the type of the switch expression.
 - A non-empty body.
- **Default clause:** At most one default clause is allowed, and it must have a body. The absence of a default clause or the presence of multiple defaults triggers an error.
- **Empty switch validation:** If no case clauses are present, a semantic error is raised.

The `insideSwitch` flag is temporarily incremented during the traversal of the `switch` body to support context-sensitive analysis (e.g., detecting whether a `break` statement is valid within the current scope).

This function ensures both structural and type correctness of switch-related constructs, contributing to reliable control-flow semantics.

checkCaseStatement

The function `checkCaseStatement` verifies the expression and body of a case statement:

```
1 static void checkCaseStatement(TreeNode* t) {
2     TreeNode* caseExpr = t->pchild;
3     TreeNode* caseBody = caseExpr ? caseExpr->psibling : NULL;
4
5     // Check case expression
6     if (caseExpr) {
7         traverse(caseExpr, nullProc, checkNode);
8         if (!isInteger(caseExpr->type)) {
9             typeError(caseExpr, "Case_must_be_integer");
10        }
11    }
12
13    // Check case body
14    if (caseBody) {
15        traverse(caseBody, nullProc, checkNode);
16    } else {
17        typeError(t, "Invalid_case_body");
18    }
19 }
```

checkIdentifier

The function `checkIdentifier` validates an identifier node:

```
1 void checkIdentifier(TreeNode* t) {
2     if (t == NULL) return;
3
4     // Ensure only called on valid identifier nodes
5     if (t->nodekind != idNK) {
```

```

6      fprintf(stderr, "[Internal_error]_checkIdentifier_called_on_
      non-idNK_node\n");
7      return;
8  }
9
10     if (t->nodeAttr.name == NULL) {
11         fprintf(stderr, "[Error]_Identifier_with_NULL_name_at_line_%d
            \n", t->lineno);
12         return;
13     }
14
15     if (t->scope == NULL) {
16         fprintf(stderr, "[Error]_currentTable_is_NULL_during_
            checkIdentifier!\n");
17         return;
18     }
19
20     BucketList entry = st_lookup(t->scope, t->nodeAttr.name);
21     if (entry == NULL) {
22         typeError(t, "Undeclared_identifier");
23     } else {
24         t->type = entry->type;
25     }
26 }

```

checkGotoStatement

The function checkGotoStatement verifies that the goto statement has a valid label:

```

1 static void checkGotoStatement(TreeNode* t) {
2     // Verify if the goto statement is valid
3     TreeNode* label = t->pchild;
4     if (label == NULL || label->nodekind != idNK) {
5         typeError(t, "goto_without_label");
6         return;

```

```

7     }
8
9     // Additional checks for label validity could be added here
10  }

```

checkNode

The `checkNode` function serves as a central dispatcher for semantic analysis. Based on the syntactic kind (`nodekind`) of a given node in the abstract syntax tree (AST), it invokes the appropriate checking routine to enforce language rules and annotate types.

```

1  void checkNode(TreeNode* t) {
2      if (t == NULL) return;
3
4      switch (t->nodekind) {
5          case opNK:
6              if (t->pchild && t->pchild->psibling)
7                  checkFunctionOperation(t);
8              else
9                  typeError(t, "Operation_missing_operands");
10             break;
11
12             case funcCallNK:
13                 if (t->pchild)
14                     checkFunctionCall(t);
15                 else
16                     typeError(t, "Function_call_missing_arguments");
17                 break;
18
19             case whileNK:
20                 checkWhileStatement(t);
21                 break;
22
23             case ifNK:
24                 checkIfStatement(t);

```



```
25         break;
26
27     case switchNK:
28         checkSwitchStatement(t);
29         break;
30
31     case caseNK:
32         checkCaseStatement(t);
33         break;
34
35     case continueNK:
36     case breakNK:
37         checkLoopControlStatement(t);
38         break;
39
40     case gotoNK:
41         checkGotoStatement(t);
42         break;
43
44     case returnNK:
45         checkReturnStatement(t);
46         break;
47
48     case idNK:
49         checkIdentifier(t);
50         break;
51
52     case numberNK:
53         t->type = intK;
54         break;
55
56     case fnumberNK:
57         t->type = floatK;
58         break;
```

```

59
60     case typeNK:
61         if (t->nodeAttr.val < voidK || t->nodeAttr.val > NONE_DK)
62             typeError(t, "Invalid_type");
63         t->type = t->nodeAttr.val;
64         break;
65
66     default:
67         // May include funcDefNK, declarNK, etc.
68         break;
69 }
70 }

```

This function ensures that each node is analyzed according to its syntactic role. It performs semantic checks such as:

- **Dispatching to specialized checkers:** Nodes like `whileNK`, `ifNK`, `switchNK`, and `funcCallNK` are routed to specific functions that verify their structure and type constraints.
- **Type inference:** For constant literals (e.g., `numberNK`, `fnumberNK`), types are inferred and assigned directly.
- **Error handling:** Nodes with incomplete or invalid structure (e.g., missing operands or arguments) trigger descriptive semantic errors.

It acts as a core building block in the semantic analysis phase by integrating both type checking and structural validation across diverse language constructs.

4.4 Code Generation

With the architectural design and operational strategies clearly defined, the next step lies in implementing the code generation logic. The implementation phase translates the design into working C functions that traverse the annotated AST and emit assembly code for the 8051 microcontroller. Each function is responsible for handling a specific class of AST nodes, ensuring that the semantics of the source program are faithfully preserved in the resulting low-level instructions. The focus during implementation

is not only on correctness, but also on maintainability and alignment with embedded system constraints, such as limited memory, register scarcity, and the need for deterministic behavior.

The `nodeBucket` variable is used in many cases to access information about a specific identifier from the symbol table. This structure stores metadata such as the identifier's type (`intK` or `charK`), scope, and storage details. By retrieving the type field from `nodeBucket`, the code generator can determine whether the variable occupies one or two bytes and emit the appropriate assembly instructions accordingly. This distinction is essential for correctly handling memory access and register usage, especially for mixed-type expressions and variable declarations. The use of `nodeBucket` ensures that the generator remains consistent with the semantic analysis phase and maintains type correctness throughout the translation process.

The following subsections describe how each component was implemented and how they interact to form a cohesive and functional code generation module.

4.4.1 Function codegen

Listing 4.27: codegen function

```
1 void codegen(TreeNode* root, FILE *out) {  
2     fout = out;  
3     emit_preamble();  
4     emit_global_variables(globalTable);  
5     linegen(root, out);  
6     emit_postamble();  
7 }
```

The `codegen()` function is the entry point for the code generation phase in the compiler. It begins by initializing the output file with the necessary memory sections for the 8051 architecture—typically `.CSEG` for code and `.DSEG` for data, using `emit_preamble` function, followed by the declaration of global variables through `emit_global_variables`. It then handles the emission of global variables and iterates over the top-level nodes of the AST (Abstract Syntax Tree), invoking the recursive `linegen()` function to begin translating the program structure into assembly code, and finally adds the ending code with `emit_postamble`.

4.4.2 Function linegen

Listing 4.28: linegen function

```
1 void linegen(TreeNode *node, FILE *out) {
2     if (node != NULL) {
3         switch (node->nodekind) {
4             case ifNK:
5             case whileNK:
6             case forNK:
7                 genStmt(node, out);
8                 linegen(node->psibling, out);
9                 break;
10            case opNK:
11            case blockNK:
12                genExp(node, out);
13                break;
14            case idNK:
15            case numberNK:
16                genExp(node, out);
17                break;
18            case returnNK:
19            case argsNK:
20            case parameterNK: {
21                genFunc(node, out);
22                break;
23            }
24            case declarNK: {
25                genStmt(node, out);
26                linegen(node->psibling, out);
27                break;
28            }
29            case funcCallNK: {
30                genFunc(node, out);
31                break;
```

```

32     }
33     case funcDefNK:
34         genFunc(node, out);
35         linegen(node->psibling, out);
36         break;
37     default:
38         break;
39 }
40 }
41 }

```

At the heart of this process is the `linegen()` function, a recursive descent code generator that traverses the AST node by node. For each node, it examines the node's kind (`nodekind`) to determine its role—whether it's a declaration, control statement, expression, or function-related construct—and dispatches the corresponding code generation routine.

This recursive structure reflects a depth-first traversal of the AST. Each function or block is processed fully before moving to the next, enabling structured and modular code emission. The `linegen()` function plays a central role here, acting as the dispatcher that ensures every node is visited and the appropriate translation function is applied.

This architecture allows the compiler backend to support arbitrarily nested constructs, including expressions inside control structures or function calls within expressions. Recursion provides a natural mechanism for handling these nests, maintaining clarity and modularity in the implementation.

Listing 4.29: *emit_global_variables function*

```

1 void emit_global_variables(SymbolTable *globalTable) {
2     emit_line(";==_Variáveis_Globais_==");
3     emit_line("DSEG_AT_30H");
4
5     // Percorre todos os buckets da tabela hash
6     for (int i = 0; i < globalTable->capacity; i++) {
7         BucketList current = globalTable->hashTable[i];
8         while (current != NULL) {
9             if (current->kind == declarNK) {
10                 fprintf(fout, "%-15s:_DS_%d\t;%s\n",

```

```

11         current->name,
12         (current->type == charK) ? 1 : 2,
13         typeKindToString(current->type));
14     }
15     current = current->next;
16 }
17 }
18
19 emit_line("CSEG");
20 emit_line("");
21 }

```

Listing 4.30: *emit_global_variables function*

```

1
2 void change_bank(FILE* out, int bank_n) {
3     int psw_bits = (bank_n & 0x03) << 3;
4
5     fprintf(out, "PUSH_A\n");
6     fprintf(out, "PUSH_PSW\n");
7
8     fprintf(out, "MOV_A, PSW\n");
9     fprintf(out, "ANL_A, #0xE7\n");
10    fprintf(out, "ORL_A, #%d\n", psw_bits);
11    fprintf(out, "MOV_PSW, A\n");
12
13    fprintf(out, "POP_PSW\n");
14    fprintf(out, "POP_A\n");
15 }

```

4.4.3 Function genFunc

The genFunc function is responsible for generating assembly code for different function-related constructs in the AST, such as function definitions, function calls, parameters, return statements, and argument lists. It uses static variables to track register usage and indexing across calls.

Listing 4.31: Function definition and prologue in genFunc

```

1  case funcDefNK:
2      TreeNode *type = node->pchild;
3      TreeNode *name = type ? type->psibling : NULL;
4      TreeNode* func_node = name ? name->psibling : NULL;
5
6      func_scope = node->scope;
7      func_name = name->nodeAttr.name;
8
9      fprintf(out, "%s:\n", name->nodeAttr.name);
10     fprintf(out, "    PUSH_B\n");
11     fprintf(out, "    PUSH_PSW\n\n");
12
13     if (has_parameters) {
14         fprintf(out, "; BANCO_1\n");
15         change_bank(out, 1);
16     }
17
18     while (func_node) {
19         linegen(func_node, out);
20         func_node = func_node->psibling;
21         if(func_node && func_node->nodekind != parameterNK) {
22             fprintf(out, "; BANCO_0\n");
23             change_bank(out, 0);
24         }
25     }
26
27     fprintf(out, "\n    POP_PSW\n");
28     fprintf(out, "    POP_B\n");
29     fprintf(out, "    RET\n");

```

When the node is a funcDefNK, it represents a function definition. The code first extracts the type, name, and body of the function by walking through its children. It checks if the function has parameters, which will require switching to register bank 1 for parameter handling. It sets global context variables

func_scope and func_name, and emits the function label, followed by prologue instructions (PUSH B, PUSH PSW) to save the state. If parameters exist, it calls change_bank(out, 1) to use bank 1. It then traverses the function body using linegen. If any node after the parameters is encountered, it switches back to bank 0 for regular execution.

Listing 4.32: Parameter handling in genFunc

```

1 case parameterNK:
2     TreeNode *ptype = node->pchild;
3     TreeNode *pname = ptype ? ptype->psibling : NULL;
4
5     int registros_livres = 8 - (parameter_index + base_reg_index);
6
7     if (ptype->nodeAttr.val == charK && registros_livres >= 1) {
8         fprintf(out, "░░░░POP░A\n");
9         fprintf(out, "░░░░MOV░R%d,░A\n", parameter_index +
10             base_reg_index);
11         parameter_index++;
12     }

```

The parameterNK case handles individual parameters. It distinguishes between char (1 byte) and int (2 bytes). For char, it pops a byte from the stack and places it in the next available register (starting at R2). For int, it would ideally pop two bytes, but this logic appears incomplete or incorrect (there's a bug in the commented POP A, @R0 line).

Listing 4.33: Return statement handling in genFunc

```

1 case returnNK: {
2     TreeNode* ret_type = node->pchild;
3     BucketList nodeBucket = st_lookup(node->scope, ret_type->nodeAttr
4         .name);
5     DeclarK tipo = nodeBucket->type;
6
7     if (tipo == intK) {
8         // código para retorno de int (não implementado aqui)
9     } else {
10         fprintf(out, ";░░░Return░da░função\n");
11         fprintf(out, "░░░░MOV░A,░%s\n", ret_type->nodeAttr.name);
12     }
13 }

```



```

11     }
12     break;
13 }

```

The returnNK case generates code to return a value. It looks up the return variable in the symbol table to identify its type. For char return values, it moves the variable's content into register A, which by convention carries the return value. The handling for int return values is currently not implemented.

Listing 4.34: Function call in genFunc

```

1 case funcCallNK:
2     TreeNode *func_id = node->pchild;
3     TreeNode *args = func_id ? func_id->psibling : NULL;
4
5     if (args) linegen(args, out);
6
7     fprintf(out, "LLLLLCALLL%s\n", func_id->nodeAttr.name);

```

In the funcCallNK case, the code prepares for a function call. It first identifies the function name and its arguments. If arguments exist, it delegates their generation to linegen(args, out). After that, it emits an LCALL instruction to jump to the function label.

Listing 4.35: Function arguments handling in genFunc

```

1 case argsNK:
2     TreeNode *arg_node = node->pchild;
3     while (arg_node) {
4         switch (arg_node->nodekind) {
5             case numberNK:
6                 linegen(arg_node, out);
7                 fprintf(out, "LLLLPUSHL\n");
8                 break;
9             case idNK:
10                BucketList var = st_lookup(arg_node->scope, arg_node
11                    ->nodeAttr.name);
12                int id_type = var->type;
13                if(id_type == charK) {
14                    linegen(arg_node, out);
15                }
16            }
17         arg_node = arg_node->psibling;
18     }

```

```

14         fprintf(out, "░░░░PUSH░A\n");
15     } else if(id_type == intK) {
16         linegen(arg_node, out);
17         fprintf(out, "░░░░PUSH░R2\n");
18         fprintf(out, "░░░░PUSH░R3\n");
19     }
20     break;
21 }
22 arg_node = arg_node->psibling;
23 }

```

The argsNK case processes function arguments before a call. For each argument (which can be a number or an identifier), it generates the corresponding expression and pushes its result onto the stack. For char, it pushes the accumulator, for int, it pushes R2 and R3.

The function handles each node type through a switch structure and emits the corresponding code. Some parts, such as int parameter and return handling, need refinement or fixing, but the overall logic correctly separates declaration, argument passing, bank switching, and call/return mechanics.

4.4.4 Function genStmt

The genStmt function generates assembly code for different types of statements in the abstract syntax tree (AST). It uses a switch statement to handle each kind of statement node.

For variable declarations, it checks if the variable is global or local. Global variables are assumed to be already declared. For local variables, it generates space in the data segment with the size depending on the variable type, prefixing the variable name with the current function name to avoid naming conflicts.

Listing 4.36: Variable declaration handling in genStmt

```

1 case declarNK: {
2     TreeNode *type = node->pchild;
3     TreeNode *name = type ? type->psibling : NULL;
4     BucketList node_b = st_lookup(name->scope, name->nodeAttr.name);
5     int dec_type = node_b->type;
6
7     if (node->scope == globalTable) {
8         fprintf(out, "░░░░Var░Global░já░Declarada\n");

```

```

9      } else {
10         if (dec_type == charK) {
11             fprintf(out, ";_Variável_local\n");
12             fprintf(out, "DSEG\n");
13             fprintf(out, "%s_%s:_DS_1\n", func_name, name->
                nodeAttr.name);
14             fprintf(out, "CSEG\n");
15         } else if (dec_type == intK) {
16             fprintf(out, ";_Variável_local\n");
17             fprintf(out, "DSEG\n");
18             fprintf(out, "%s_%s:_DS_2\n", func_name, name->nodeAttr.
                name);
19             fprintf(out, "CSEG\n");
20         }
21     }
22     break;
23 }

```

For control flow statements like if, while, and for loops, the function generates labels to manage branching and looping. It evaluates the condition and uses jump instructions to decide which part of the code to execute next. For example, in an if statement, it jumps over the then-block if the condition is false, optionally executing an else-block if present. Loops are handled by creating start and end labels, jumping back to the start as long as the loop condition remains true.

Listing 4.37: If statement handling in genStmt

```

1  case ifNK: {
2      TreeNode *condNode = node->pchild;
3      TreeNode *thenNode = condNode ? condNode->psibling : NULL;
4      condNode = condNode->pchild;
5      TreeNode *elseNode = thenNode ? thenNode->psibling : NULL;
6      elseNode = elseNode ? elseNode->pchild : NULL;
7
8      static int labelCount = 0;
9      int falseLabel = labelCount++;
10     int endLabel = labelCount++;

```

```

11
12     fprintf(out, "    ;verificação_if\n");
13     linegen(condNode, out);
14     fprintf(out, "    JZ_IF_L%d\n", falseLabel);
15     if (elseNode)
16         fprintf(out, "    SJMP_IF_L%d\n", endLabel);
17     fprintf(out, "IF_L%d:    ;THEN\n", falseLabel);
18     linegen(thenNode, out);
19     fprintf(out, "    SJMP_end_if\n");
20     if (elseNode) {
21         fprintf(out, "IF_L%d:    ;ELSE\n", endLabel);
22         linegen(elseNode, out);
23     }
24     fprintf(out, "end_if\n");
25     break;
26 }

```

Each case handles the statement's specific structure, recursively calling `linegen` on child nodes to generate code for expressions or nested statements.

Overall, `genStmt` converts the logical structure of high-level statements into the low-level assembly instructions and labels necessary to preserve their flow and semantics in the target machine code.

4.4.5 Function `genExp`

The `genExp` function generates assembly code for expressions represented in the abstract syntax tree (AST). It handles different kinds of expression nodes and produces the corresponding instructions to evaluate them on the target machine.

Listing 4.38: Handling numeric constants in `genExp`

```

1 case numberNK: {
2     fprintf(out, "    ;Constant_value\n");
3     fprintf(out, "    MOV_A,_%#d;load_constant_into_accumulator\n",
4             node->nodeAttr.val);
5     break;
6 }

```

For numeric constants (numberNK), it simply loads the constant value into the accumulator, when char, and into R2 and R3 when integer.

Listing 4.39: Handling identifier access in genExp

```

1 case idNK: {
2     BucketList nodeBucket = st_lookup(node->scope, node->
3         nodeAttr.name);
4     DeclaraK tipo = nodeBucket->type;
5     fprintf(out, ";Identificador\n", node->nodeAttr.name);
6     if(node->scope == globalTable) {
7         if(tipo == intK) {
8             if(registros_livres >= 2) {
9                 fprintf(out, "MOV_R0,%s\n", node->
10                     nodeAttr.name);
11                 for(int i = 0; i < 2; i++){
12                     fprintf(out, "MOV_A,@R0\n");
13                     fprintf(out, "MOV_R%d,A\n",
14                         base_reg_index1+variables_index);
15                     fprintf(out, "INC_R0\n");
16                     variables_index++;
17                 }
18             }
19         } else if(tipo == charK) {
20             fprintf(out, "MOV_A,%s\n", node->nodeAttr.
21                 name);
22         }
23     } else if(func_scope == node->scope) {
24         if(tipo == intK) {
25             if(registros_livres >= 2) {
26                 fprintf(out, "MOV_R0,%s_%s\n", node->
27                     nodeAttr.name);
28                 for(int i = 0; i < 2; i++){
29                     fprintf(out, "MOV_A,@R0\n");
30                     fprintf(out, "MOV_R%d,A\n",
31                         base_reg_index1+variables_index);

```

```

26         fprintf(out, "▯▯▯▯INC▯R0▯\n");
27         variables_index++;
28     }
29 }
30 } else if(tipo == charK) {
31     fprintf(out, "▯▯▯▯MOV▯A,▯%s_▯s▯\n", func_name ,
32         node->nodeAttr.name);
33 }
34 break;
35 }

```

For identifiers (idNK), the function looks up the variable in the symbol table to determine its type and scope. It then generates the code to load the variable's value into registers and adapted in case of local variables. The handling differs based on whether the variable is global or local and whether it is an integer (which takes 2 bytes and requires loading into two registers) or a char (1 byte, loaded into the accumulator). The function tracks how many registers are already in use to avoid register overflow.

For blocks (blockNK), it recursively generates code for the statements contained within the block.

Listing 4.40: Handling compound blocks in genExp

```

1 case blockNK: {
2     TreeNode *stmt = node->pchild;
3     linegen(stmt, out);
4     break;
5 }

```

For operators (opNK), the function identifies the operator type and generate the corresponding assembly instructions to perform the operation.

Listing 4.41: Handling binary operations in genExp

```

1 case opNK: {
2     TreeNode *left = node->pchild;
3     TreeNode *right = left ? left->psibling : NULL;
4     int op = node->nodeAttr.val;
5     switch (op) {
6         case PLUS: {

```

```

7          // Generate code for left and right, then perform
          addition
8          break;
9      }
10     case MINUS: {
11         // Subtract right from left
12         break;
13     }
14     // Other operations (MUL, DIV, comparisons, etc.)
15     default:
16         break;
17 }
18 break;
19 }

```

Overall, genExp converts the high-level expression tree into low-level assembly code that computes the expression's value, carefully managing registers and respecting variable scopes.

Chapter 5

Tests and Validation

This chapter presents the various tests and validation procedures conducted to ensure the correctness and reliability of the C compiler targeting the ISA 8051 architecture. Rigorous testing is essential to verify that each stage of the compilation process—from parsing and abstract syntax tree (AST) generation to semantic analysis and final code generation—operates as intended and produces accurate results. The sections that follow describe the testing methodologies employed, including AST tests to verify the syntactic structure, semantic tests to check for logical consistency and correctness, and dry runs that simulate the compiler’s execution to validate the overall compilation flow. These tests collectively demonstrate the robustness and functionality of the compiler, providing confidence in its practical use for embedded system development.

5.1 AST Test

This chapter presents the various tests and validation procedures conducted to ensure the correctness and reliability of the C compiler targeting the ISA 8051 architecture. Rigorous testing is essential to verify that each stage of the compilation process—from parsing and abstract syntax tree (AST) generation to semantic analysis and final code generation—operates as intended and produces accurate results. The sections that follow describe the testing methodologies employed, including AST tests to verify the syntactic structure, semantic tests to check for logical consistency and correctness, and dry runs that simulate the compiler’s execution to validate the overall compilation flow. These tests collectively demonstrate the robustness and functionality of the compiler, providing confidence in its practical use for embedded system development.

Listing 5.1: Test Code

```
1
2 #define LIMIT 5
3 const int factor;
4
```



```
5 int multiply(int a, int b) {
6     return a * b;
7 }
8
9 int main() {
10     int values[5] = {1, 2, 3, 4, 5};
11     int result = 0;
12
13     for (int i = 0; i < LIMIT; i++) {
14         result += multiply(2, factor);
15     }
16
17     return result;
18 }
```

```

#define
  idNK: LIMIT
  numberNK: 5
DeclarNK
  Qualifier: const
  Tipo: int
  idNK: factor
funcDefNK
  Tipo: int
  idNK: multiply
  parameterNK
    Tipo: int
    idNK: a
  parameterNK
    Tipo: int
    idNK: b
  blockNK:
    returnNK:
      opNK: *
      idNK: a
      idNK: b
funcDefNK
  Tipo: int
  idNK: main
  blockNK:
    opNK: =
    DeclarNK
      arrayNK
        Tipo: int
        idNK: values
        indexNK
          numberNK: 5
        numberNK: 1
        numberNK: 2
        numberNK: 3
        numberNK: 4
        numberNK: 5
    opNK: =
    DeclarNK
      Tipo: int
      idNK: result
      numberNK: 0
    For:
      opNK: =
      DeclarNK
        Tipo: int
        idNK: i
        numberNK: 0
      opNK: <
      idNK: i
      idNK: LIMIT

```

Figure 25: Generated Tree

```

opNK: ++
  idNK: i
blockNK:
  opNK: =
  idNK: result
  opNK: +
  idNK: result
  funcCallNK
    idNK: multiply
    argsNK
      numberNK: 2
      idNK: factor
  returnNK:
    idNK: result

```

Figure 26: Generated Tree

Listing 5.2: Test Code

```

1
2 #define STEPS 4
3 const int increment = 3;
4
5 int add(int x, int y) {
6     return x + y;
7 }
8

```

```

9  int main() {
10     const int start = 10;
11     int current = start;
12     int sequence[STEPS] = {0};
13     int i = 0;
14
15     while (i < STEPS) {
16         sequence[i] = current;
17         i++;
18     }
19
20     // Após o ciclo, um if separado para fazer uma verificação
       simples
21     if (sequence[STEPS - 1] > 15) {
22         current = add(current, increment); // Exemplo de ação
           condicional
23     }
24
25     return sequence[STEPS - 1];
26 }

```

```

#define
  idNK: STEPS
  numberNK: 4
DeclarNK
  Qualifier: const
  Tipo: int
  idNK: increment
funcDefNK
  Tipo: int
  idNK: add
  parameterNK
    Tipo: int
    idNK: x
  parameterNK
    Tipo: int
    idNK: y
  blockNK:
    returnNK:
      opNK: +
      idNK: x
      idNK: y
funcDefNK
  Tipo: int
  idNK: main
  blockNK:
    opNK: =
    DeclarNK
      Qualifier: const
      Tipo: int
      idNK: start
      numberNK: 10
    opNK: =
    DeclarNK
      Tipo: int
      idNK: current
      idNK: start
    opNK: =
    DeclarNK
      arrayNK
        Tipo: int
        idNK: sequence
        indexNK
          numberNK: 2
        numberNK: 0
    opNK: =
    DeclarNK
      Tipo: int
      idNK: i
      numberNK: 0
  whileNK
    opNK: <
    idNK: i
    idNK: STEPS
    blockNK:
      opNK: =

```

Figure 27: Generated Tree

```

      arrayAccessNK
        idNK: sequence
        indexNK
          idNK: i
        idNK: current
      opNK: ++
      idNK: i
If:
  Then:
    opNK: >
    arrayAccessNK
      idNK: sequence
      indexNK
        numberNK: 7
      numberNK: 15
    blockNK:
      opNK: =
      idNK: current
      funcCallNK
        idNK: add
        argsNK
          idNK: current
          idNK: incremer
    returnNK:
      arrayAccessNK
        idNK: sequence
        indexNK
          numberNK: 1

```

Figure 28: Generated Tree

5.2 Semantic Test

The examples below illustrate typical test cases and the corresponding semantic checks:

Redeclaration Detection This test aims to validate the correct detection of variable redeclaration within the same scope. In C, declaring the same identifier more than once in a single block is not allowed and should be flagged as a semantic error. The semantic analyzer traverses the abstract syntax tree (AST) and attempts to insert each variable into the symbol table associated with the current scope. Upon encountering the second declaration of `x`, it uses the `st_lookupLocal` function to check if this identifier already exists in that scope. As it does, the compiler emits an error message. The output presented in Figure 29 clearly shows the detection, confirming that the scope and symbol management system is working as intended. This mechanism is essential to prevent ambiguities and ensure compliance with the C standard.

```
1 int x;  
2 int x; // Error: Symbol already declared in this scope.
```

```
[ Análise Semântica ] Construção da Tabela de Símbolos  
[insertNode] Case declarNK: sibling: declarNK, child: typeNK  
[DEBUG] A tentar inserir 'x' (linha 0) no scope 0  
[lookupLocal] A procurar 'x' apenas no scope 0...  
!!!! DEBUGDEBUGDEBUGDEBUGDEBUGDEBUG !!!!  
[lookupLocal] A procurar 'x' apenas no scope 0...  
[insertNode] A sair de case declarNK  
[lookup] A procurar 'x' do scope 0 para cima...  
Símbolo encontrado  
[insertNode] Case declarNK: sibling: nenhum, child: typeNK  
[DEBUG] A tentar inserir 'x' (linha 0) no scope 0  
[lookupLocal] A procurar 'x' apenas no scope 0...  
  
Erro: variável 'x' já declarada neste scope.  
  
[insertNode] A sair de case declarNK  
[lookup] A procurar 'x' do scope 0 para cima...  
Símbolo encontrado  
  
>>> Tabela de Símbolos Global:
```

Figure 29: Compiler output showing detection of redeclaration of a variable within the same scope.

Undeclared Identifier Usage This test validates the correct detection of the use of an undeclared variable. In C, all variables must be declared before use; otherwise, the compiler should produce a semantic error. In this example, the identifier `y` is used in an expression without a prior declaration. The

abstract syntax tree is traversed and, upon encountering the use of `y`, searches for this identifier in the symbol tables associated with the current and enclosing scopes. As it is not found, the compiler emits the appropriate error message. The output in Figure 30 demonstrates that the semantic analysis correctly prevents the use of undefined identifiers.

```
1 int main() {  
2     y = 5; // Error: 'y' undeclared.  
3     return 0;  
4 }
```

```
[ Análise Semântica ] Construção da Tabela de Símbolos  
[lookupLocal] A procurar 'main' apenas no scope 0...  
!!!! DEBUGDEBUGDEBUGDEBUGDEBUGDEBUG !!!!  
[lookupLocal] A procurar 'main' apenas no scope 0...  
[insertNode] A sair de case funcDefNK  
  
Novo nível de scope: 1  
  
[lookup] A procurar 'main' do scope 1 para cima...  
[lookup] A procurar 'main' do scope 0 para cima...  
Símbolo encontrado  
[insertNode] Case opNK: sibling: returnNK, child: idNK  
[lookup] A procurar 'y' do scope 1 para cima...  
[lookup] A procurar 'y' do scope 0 para cima...  
Símbolo não encontrado.  
  
Erro: identificador 'y' usado mas não declarado previamente (linha 0).  
  
Scope restaurado!  
  
[lookup] A procurar 'main' do scope 0 para cima...  
Símbolo encontrado
```

Figure 30: Compiler output showing detection of the use of an undeclared variable.

Scope Handling This test demonstrates the correct implementation of variable shadowing rules, where a variable declared in an inner block can hide a variable of the same name declared in an outer block. In C, this allows different scopes to contain variables with the same identifier, without conflict, and always resolves to the nearest declaration. In the example, the variable `x` is declared both in the global scope and in a nested block. The compiler correctly manages the symbol tables, ensuring that each declaration of `x` is inserted into the appropriate scope and that symbol lookups respect the scope hierarchy. The output in Figure 31 illustrates this mechanism in action.

```
1 int x = 1;  
2 void func() {  
3     int x = 2; // Shadows the outer 'x'
```

```

4 // 'x' here refers to the inner declaration
5 }

```

```

[ Análise Semântica ] Construção da Tabela de Símbolos
[insertNode] Case opNK: sibling: funcDefNK, child: declarNK
[insertNode] Case declarNK: sibling: numberNK, child: typeNK
[DEBUG] A tentar inserir 'x' (linha 0) no scope 0
[lookupLocal] A procurar 'x' apenas no scope 0...
!!!! DEBUGDEBUGDEBUGDEBUGDEBUGDEBUG!!!!
[lookupLocal] A procurar 'x' apenas no scope 0...
[insertNode] A sair de case declarNK
[lookup] A procurar 'x' do scope 0 para cima...
Símbolo encontrado
[lookupLocal] A procurar 'func' apenas no scope 0...
!!!! DEBUGDEBUGDEBUGDEBUGDEBUGDEBUG!!!!
[lookupLocal] A procurar 'func' apenas no scope 0...
[insertNode] A sair de case funcDefNK

Novo nível de scope: 1

[lookup] A procurar 'func' do scope 1 para cima...
[lookup] A procurar 'func' do scope 0 para cima...
Símbolo encontrado
[insertNode] Case opNK: sibling: nenhum, child: declarNK
[insertNode] Case declarNK: sibling: numberNK, child: typeNK
[DEBUG] A tentar inserir 'x' (linha 0) no scope 1
[lookupLocal] A procurar 'x' apenas no scope 1...
!!!! DEBUGDEBUGDEBUGDEBUGDEBUGDEBUG!!!!
[lookupLocal] A procurar 'x' apenas no scope 1...
[insertNode] A sair de case declarNK
[lookup] A procurar 'x' do scope 1 para cima...
Símbolo encontrado

Scope restaurado!

[lookup] A procurar 'func' do scope 0 para cima...
Símbolo encontrado

>>> Tabela de Símbolos Global:

```

Figure 31: Compiler output showing correct management of shadowed variables in nested scopes.

Function Declaration and Definition Mismatch This test aims to validate that the compiler detects inconsistencies between a function's declaration and its definition. According to the C standard, a function definition must match a previous declaration exactly in both return type and parameter types. In the example, the function `foo` is declared with one return type and defined with another, causing the compiler to emit the appropriate error. The output in Figure 32 confirms that the mechanism for checking function signatures is working correctly and enforcing compliance with the language rules.

Listing 5.3: Example of C code with inconsistent function declaration and definition

```

1 int foo(int x);           // Declaration: returns int
2 float foo(float y) {     // Definition: returns float, parameter is float
3     return y;
4 }

```

```

[ Análise Semântica ] Construção da Tabela de Símbolos
[lookupLocal] A procurar 'foo' apenas no scope 0...
!!!! DEBUGDEBUGDEBUGDEBUGDEBUGDEBUG !!!!
[lookupLocal] A procurar 'foo' apenas no scope 0...
[insertNode] A sair de case funcDeclNK
[lookupLocal] A procurar 'foo' apenas no scope 0...

Erro: definição da função 'foo' não coincide com a declaração anterior - Tipo de retorno diferente.

[insertNode] A sair de case funcDefNK

Novo nível de scope: 1

[lookup] A procurar 'foo' do scope 1 para cima...
[lookup] A procurar 'foo' do scope 0 para cima...
Símbolo encontrado
[insertNode] Case parameterNK: sibling: blockNK, child: typeNK
[DEBUG] A tentar inserir 'y' (linha 0) no scope 1
[lookupLocal] A procurar 'y' apenas no scope 1...
!!!! DEBUGDEBUGDEBUGDEBUGDEBUGDEBUG !!!!
[lookupLocal] A procurar 'y' apenas no scope 1...
[insertNode] A sair de case parameterNK
[lookup] A procurar 'y' do scope 1 para cima...
Símbolo encontrado
[lookup] A procurar 'y' do scope 1 para cima...
Símbolo encontrado

Scope restaurado!

[lookup] A procurar 'foo' do scope 0 para cima...
Símbolo encontrado

>>> Tabela de Símbolos Global:

```

Figure 32: Compiler output showing detection of a mismatch between function declaration and definition.

Use of break Outside a Loop or Switch This test validates that the compiler correctly detects the use of the `break` statement outside of a valid context, that is, outside of a loop (`for`, `while`) or a `switch` statement. According to the C standard, such usage constitutes a semantic error. In the example, the `break` command is used inside the body of a function, but not within any loop or `switch`. The compiler traverses the abstract syntax tree, checks the context of the statement, and, since it is not inside a valid construct, emits the appropriate error message. The output in Figure 33 demonstrates that the semantic analysis is working correctly for this case.

Listing 5.4: Example of C code using `break` outside a loop or switch

```

1 void foo() {
2     break; // Error: break outside of loop or switch
3 }

```



```

[ Análise Semântica ] Verificação de tipos
[lookup] A procurar 'foo' do scope 0 para cima...
Símbolo encontrado

SCOPE Nó typeNK: 1
[checkNode] nodekind = typeNK (tipo: voidK)
[checkNode] pchild é NULL
[checkNode] tem psibling (nodekind = 18)

SCOPE Nó idNK: 1
[checkNode] nodekind = idNK (id: foo)
[checkNode] pchild é NULL
[checkNode] tem psibling (nodekind = 22)
[lookup] A procurar 'foo' do scope 1 para cima...
[lookup] A procurar 'foo' do scope 0 para cima...
Símbolo encontrado

SCOPE Nó breakNK: 1
[checkNode] nodekind = breakNK
[checkNode] pchild é NULL
[TypeError] Linha 0: break fora de um loop ou switch

SCOPE Nó blockNK: 1
[checkNode] nodekind = blockNK
[checkNode] pchild nodekind = breakNK
[WARNING] Tipo de nó não tratado: 22

SCOPE Nó funcDefNK: 1
[checkNode] nodekind = funcDefNK
[checkNode] pchild nodekind = typeNK

[ Compilador ] Análise terminada com sucesso.

```

Figure 33: Compiler output showing detection of the use of break outside a loop or switch.

Overall, the module effectively detects common semantic violations while accurately propagating scope and type information throughout the Abstract Syntax Tree (AST).

5.3 Dry Run

In the following subsections, we describe the implementation of code generation for a representative set of AST node types. For each case, we present a snippet of the corresponding assembly code and explain how it was generated, detailing how it reflects the semantics of the original C construct and adheres to the architectural and memory constraints defined earlier.

Listing 5.5: Code test for logical and arithmetic operations

```

1 char a = 5;

```

```

2 char b;
3
4 void main() {
5     b = a & 3;
6     b = a + 2;
7     b = a ^ b;
8 }

```

```

1 ORG 0x0000
2 SJMP START
3
4 ORG 0x0030
5 START:
6
7 ; === Variáveis Globais ===
8 DSEG AT 30H
9 a          : DS 1    ; charK
10 b          : DS 1    ; charK
11 CSEG
12
13 ; Constante
14 MOV A, #5 ; constante
15 MOV a, A ; atribuição
16 ; Var Global já Declarada
17 main:
18     PUSH B
19     PUSH PSW
20
21 ; Constante
22 MOV A, #3 ; constante
23 MOV R0, A
24 ; Identificador
25 MOV A, a
26 ANL A, R0
27 MOV main_b, A ; atribuição

```

```

28 ;   Constante
29     MOV A, #2 ; constante
30     MOV R0, A ; salvar operando esquerdo
31 ;   Identificador
32     MOV A, a
33     ADD A, R0 ; A = esquerda + direita
34     MOV main_b, A ; atribuição
35 ;   Identificador
36     MOV A, b
37     MOV R0, A
38 ;   Identificador
39     MOV A, a
40     XRL A, R0
41     MOV main_b, A ; atribuição
42
43     POP PSW
44     POP B
45     RET
46
47 SJMP $
48 END

```

Listing 5.6: Generated code test for logical and arithmetic operations

This example illustrates code generation for basic arithmetic and logical operations on char variables. In the C code, variable `a` is initialized to 5, and `b` stores the results of three operations: bitwise AND, addition, and bitwise XOR. The generated assembly uses the accumulator and auxiliary registers to perform operations. Each instruction: `ANL`, `ADD`, and `XRL` corresponds directly to the C operations, with results stored in `b`.

Listing 5.7: Code test for while

```

1 char a = 0;
2 void main() {
3     while(a<3){
4         a=a+2;

```

```
5     }  
6 }
```

```
1  ORG 0x0000  
2  SJMP START  
3  
4  ORG 0x0030  
5  START:  
6  
7  ; === Variáveis Globais ===  
8  DSEG AT 30H  
9  a          : DS 1    ; charK  
10 CSEG  
11  
12 ;   Constante  
13   MOV A, #0 ; constante  
14   MOV a, A ; atribuição  
15 main:  
16   PUSH B  
17   PUSH PSW  
18  
19 WHILE_BEGIN0:  
20 ; condição  
21 ;   Constante  
22   MOV A, #3 ; constante  
23   MOV R0, A  
24 ;   Identificador  
25   MOV A, a  
26   CLR C  
27   SUBB A, R0  
28   JC Tcond0  
29   JZ Fcond0  
30 _Fcond0:  
31   MOV A, #0  
32   SJMP end0
```

```

33 Tcond0:
34     MOV A, #1
35 end0:
36     JZ WHILE_ENDO ; se condição == 0, sai do laço
37 ; corpo
38 ; Constante
39     MOV A, #2 ; constante
40     MOV R0, A ; salvar operando esquerdo
41 ; Identificador
42     MOV A, a
43     ADD A, R0 ; A = esquerda + direita
44     SJMP WHILE_BEGIN0
45 WHILE_ENDO:
46
47     POP PSW
48     POP B
49     RET
50
51 SJMP $
52 END

```

Listing 5.8: Generated code test for while

This example demonstrates the code generation process for a while loop in C operating on a char variable. In the C code, variable `a` is initialized to 0, and the loop continues while `a < 3`, incrementing `a` by 2 in each iteration. In the generated assembly, the condition is evaluated using a SUBB instruction with a carry check (JC) to simulate the comparison `a < 3`. The result of the comparison is stored in the accumulator and checked with JZ (jump if zero) to exit the loop when the condition is false. The loop body performs the addition using ADD and loops back to the condition check with SJMP. This example shows how high-level control structures like while are translated into conditional jumps and labels (WHILE_BEGIN0, WHILE_END0), maintaining correct logic flow within the constraints of assembly.

Listing 5.9: Code test for if

```

1 char a = 1;
2 char b = 2;

```

```

3 char c = 3;
4
5 void main() {
6     if (a==b){
7         ++c;
8     }else{
9         --c;
10    }
11 }

```

```

1  ORG 0x0000
2  SJMP START
3
4  ORG 0x0030
5  START:
6
7  ; === Variáveis Globais ===
8  DSEG AT 30H
9  a          : DS 1    ; charK
10 b          : DS 1    ; charK
11 c          : DS 1    ; charK
12 CSEG
13
14 ;   Constante
15 MOV A, #1 ; constante
16 MOV a, A ; atribuição
17 ;   Constante
18 MOV A, #2 ; constante
19 MOV b, A ; atribuição
20 ;   Constante
21 MOV A, #3 ; constante
22 MOV c, A ; atribuição
23 main:
24 PUSH B
25 PUSH PSW

```

```

26
27     ;verificação if
28 ;   Identificador
29     MOV A, b
30     MOV R0, A
31 ;   Identificador
32     MOV A, a
33     CLR C
34     SUBB A, R0
35     JNZ Fcond0
36     MOV A, #1
37     SJMP end0
38 Fcond0:
39     MOV A, #0
40 end0:
41     JZ IF_L0
42     SJMP IF_L1
43 IF_L0:      ; THEN
44 ;   Identificador
45     MOV A, c
46     INC A
47     MOV c, A ; guarda novo valor em c
48     SJMP end_if
49 IF_L1:      ; ELSE
50 ;   Identificador
51     MOV A, c
52     INC A
53     MOV c, A ; guarda novo valor em c
54 end_if
55
56     POP PSW
57     POP B
58     RET
59

```

```

60 SJMP $
61 END

```

Listing 5.10: Generated code test for if

This example illustrates how an if-else statement is translated from C to assembly code. The C code defines three char variables a, b, and c. In main(), it checks if a == b; if true, it increments c, otherwise it decrements c. In the assembly code, the equality check a == b is performed by subtracting b from a using the SUBB instruction and checking the zero flag with JNZ to decide the branch. Two labels, IF_L0 (then branch) and IF_L1 (else branch), control the flow based on this condition. The increment (INC) and assignment to c happen inside the corresponding branches. Finally, a jump to end_if ensures proper control flow after the conditional.

Listing 5.11: Code test for local and globals variables

```

1
2 char a = 1;
3 char b = 2;
4 char c = a + b;
5
6 void main() {
7     char d;
8     d = 5;
9
10 }

```

```

1 ORG 0x0000
2 SJMP START
3
4 ORG 0x0030
5 START:
6
7 ; === Variáveis Globais ===
8 DSEG AT 30H
9 a          : DS 1    ; charK
10 b          : DS 1    ; charK

```



```

11 c                : DS 1    ; charK
12 CSEG
13
14 ; Constante
15     MOV A, #1 ; constante
16     MOV a, A ; atribuição
17 ; Constante
18     MOV A, #2 ; constante
19     MOV b, A ; atribuição
20 ; Identificador
21     MOV A, b
22     MOV R0, A ; salvar operando esquerdo
23 ; Identificador
24     MOV A, a
25     ADD A, R0 ; A = esquerda + direita
26     MOV c, A ; atribuição
27 main:
28     PUSH B
29     PUSH PSW
30
31 ; Variável local
32 DSEG
33     main_d: DS 1
34 CSEG
35 ; Constante
36     MOV A, #5 ; constante
37     MOV main_d, A ; atribuição
38
39     POP PSW
40     POP B
41     RET
42
43 SJMP $
44 END

```

Listing 5.12: Generated code test for local and globals variables

This example demonstrates handling of both global and local variables. In the C code, three global variables a, b, and c are declared and initialized at the global scope, with c initialized as the sum of a and b. Inside main(), a local variable d is declared and assigned the value 5. The assembly code reflects this by allocating global variables a, b, and c in the data segment (DSEG at 0x30), initializing a and b with constants, then computing $c = a + b$ using register R0 as a temporary storage for one operand before performing the addition and storing the result in c. Within main(), space for the local variable d is allocated separately (also using DSEG), and it is assigned the constant value 5. The use of PUSH and POP instructions around the function body preserves registers across the function call. This code illustrates the clear distinction and management of global and local variables in memory, along with the initialization and arithmetic operations needed for the global initializers.

Listing 5.13: Code test for return

```
1
2 char a = 1;
3 char b = 2;
4 char c = a + b;
5
6 char main() {
7     char d;
8     d = 5;
9
10    return d;
11 }
```

```
1 ORG 0x0000
2 SJMP START
3
4 ORG 0x0030
5 START:
6
7 ; === Variáveis Globais ===
```

```

8 DSEG AT 30H
9 a          : DS 1    ; charK
10 b          : DS 1    ; charK
11 c          : DS 1    ; charK
12 CSEG
13
14 ; Constante
15 MOV A, #1 ; constante
16 MOV a, A ; atribuição
17 ; Constante
18 MOV A, #2 ; constante
19 MOV b, A ; atribuição
20 ; Identificador
21 MOV A, b
22 MOV R0, A ; salvar operando esquerdo
23 ; Identificador
24 MOV A, a
25 ADD A, R0 ; A = esquerda + direita
26 MOV c, A ; atribuição
27 main:
28 PUSH B
29 PUSH PSW
30
31 ; Variável local
32 DSEG
33 main_d: DS 1
34 CSEG
35 ; Constante
36 MOV A, #5 ; constante
37 MOV main_d, A ; atribuição
38 ; Return da função
39 MOV A, d
40
41 POP PSW

```

```

42     POP B
43     RET
44 ola:
45     PUSH B
46     PUSH PSW
47
48 ;   Variável local
49 DSEG
50     ola_e: DS 1
51 CSEG
52
53     POP PSW
54     POP B
55     RET
56
57 SJMP $
58 END

```

Listing 5.14: Generated code test for return

This example illustrates the implementation of a function with a return value in the generated assembly code. In the C code, global variables `a`, `b`, and `c` are declared and initialized similarly to previous examples. The function `main` declares a local variable `d`, assigns it the value 5, and returns `d`. The assembly code follows the same pattern for initializing global variables and computing the sum for `c`. Inside `main`, space is allocated for the local variable `d` and it is assigned the constant value 5. The key difference here is the handling of the return value: before exiting, the value of `d` is moved into the accumulator register `A`, which conventionally holds the return value in this calling convention for 8051. Finally, the function restores the preserved registers (`PSW` and `B`) and executes the `RET` instruction to return control to the caller, with the return value ready in `A`. This demonstrates how the compiler manages local variables and function return values in assembly for the 8051 architecture.

Listing 5.15: Code test for char and int variables

```

1
2 int a=0;
3 char b=0;

```

```

4
5 void main(){
6     ++a;
7     ++b;
8 }

```

```

1  ORG 0x0000
2  SJMP START
3
4  ORG 0x0030
5  START:
6
7  ; === Variáveis Globais ===
8  DSEG AT 30H
9  a                : DS 2    ; intK
10 b                : DS 1    ; charK
11 CSEG
12
13 ; Constante
14 MOV A, #0 ; constante
15 MOV a, A ; atribuição declarNK
16 ; Constante
17 MOV A, #0 ; constante
18 MOV b, A ; atribuição declarNK
19 main:
20 PUSH B
21 PUSH PSW
22
23 ; Identificador
24 MOV R0, a
25 MOV A, @R0
26 MOV R1, A
27 INC R0
28 MOV A, @R0
29 MOV R2, A

```

```

30     INC R0
31     INC A
32     MOV a, A ; guarda novo valor em a
33
34     POP PSW
35     POP B
36     RET
37
38 SJMP $
39 END

```

Listing 5.16: Generated code test for char and int variables

This example demonstrates how the code generator handles global variables of different types—int and char—and their incrementation within the main function.

In the C source, the integer variable `a` and the character variable `b` are both initialized to 0. The function `main` then increments both using the prefix increment operator (`++`).

In the generated assembly code, memory is allocated accordingly: `a` uses two bytes (DS 2) to represent a 16-bit int, while `b` uses one byte (DS 1) for a char. Initialization is performed by moving 0 into the corresponding memory addresses.

The incrementation of `a` is more involved because it spans two bytes. The code loads both bytes using indirect addressing (`@R0`), then performs the increment on the least significant byte and stores the result back to memory. This demonstrates the code generator's support for multi-byte operations and the distinction in handling data types.

The increment of `b` is omitted in the current output—possibly due to an issue in the code generator—but would normally involve a simple `MOV`, `INC`, and `MOV` sequence similar to the `++c` example shown earlier.

Listing 5.17: Code test for function call, parameters and return

```

1
2 char soma(char a, char b){
3     char x = a+b;
4     return x;
5 }
6

```

```

7 void main(){
8     char a=1;
9     char b=3;
10    char c;
11    c=soma(a,b);
12 }

```

```

1  ORG 0x0000
2  SJMP START
3
4  ORG 0x0030
5  START:
6
7  ; === Variáveis Globais ===
8  DSEG AT 30H
9  CSEG
10
11 soma:
12     PUSH B
13     PUSH PSW
14
15 ;   banco 1
16     PUSH A
17     PUSH PSW
18     MOV A, PSW
19     ANL A, #0xE7
20     ORL A, #8
21     MOV PSW, A
22     POP PSW
23     POP A
24 ;   parametros da função
25     POP A
26     MOV R2, A
27 ;   parametros da função
28     POP A

```

```

29     MOV R3, A
30 ;     banco 0
31     PUSH A
32     PUSH PSW
33     MOV A, PSW
34     ANL A, #0xE7
35     ORL A, #0
36     MOV PSW, A
37     POP PSW
38     POP A
39 ;     Identificador
40     MOV A, soma_b
41     MOV R0, A ; salvar operando esquerdo
42 ;     Identificador
43     MOV A, soma_a
44     ADD A, R0 ; A = esquerda + direita
45 ;     Variável local
46     DSEG
47     soma_x: DS 1
48     CSEG
49 ;     Identificador
50     MOV A, soma_b
51     MOV R0, A ; salvar operando esquerdo
52 ;     Identificador
53     MOV A, soma_a
54     ADD A, R0 ; A = esquerda + direita
55     MOV soma_x, A ; atribuição
56 ;     Return da função
57     MOV A, x
58
59     POP PSW
60     POP B
61     RET
62 main:

```



```

63     PUSH B
64     PUSH PSW
65
66 ;   Constante
67     MOV A, #1 ; constante
68 ;   Variável local
69     DSEG
70     main_a: DS 1
71     CSEG
72 ;   Constante
73     MOV A, #1 ; constante
74     MOV main_a, A ; atribuição
75 ;   Constante
76     MOV A, #3 ; constante
77 ;   Variável local
78     DSEG
79     main_b: DS 1
80     CSEG
81 ;   Constante
82     MOV A, #3 ; coqnstante
83     MOV main_b, A ; atribuição
84 ;   Chamar Função
85 ;   argumentos da função
86 ;   Identificador
87     MOV A, main_a
88     PUSH A
89 ;   Identificador
90     MOV A, main_b
91     PUSH A
92     LCALL soma
93
94     POP PSW
95     POP B
96     RET

```

```
97  
98 SJMP $  
99 END
```

Listing 5.18: Generated code test for function call, parameters and return

This test demonstrates function calls with parameters and return values. The function `soma` takes two `char` arguments, adds them, and returns the result. In `main`, two local variables are initialized and passed to `soma`, with the result stored in a third variable. The generated assembly includes register bank switching for parameter handling, stack usage for passing arguments, and return value management via the accumulator (A). Local variables and function parameters are allocated properly. This example validates the correct implementation of function call conventions in the code generator.

Throughout the development and testing of the code generation module, we validated the behavior of various AST node kinds by writing representative C-like programs and observing the emitted assembly code. Each code generation path was triggered by recursive traversal of the AST, ensuring that nested structures, such as expressions inside control statements or function calls within expressions, were handled correctly.

The recursive nature of `linegen()` and its helper functions (`genStmt`, `genExp`, `genFunc`) allowed us to manage complex constructs in a structured and hierarchical manner. For example, blocks and control flow statements like `if`, `while`, and `for` were recursively decomposed into sub-statements, and each part of the tree was processed in post-order, allowing child expressions to be fully evaluated before emitting code for the parent operation.

We tested constant expressions, variable access, arithmetic operations, logical conditions, blocks, function calls, and return statements. In each case, we verified that the correct number of instructions was generated, and that the semantics of the original source program were preserved in the output assembly.

Local and global variables were handled carefully based on scope. Global variables were allocated in the data segment and accessed directly by name. Local variables were prefixed with the function name (e.g., `func_varname`) and reserved space using `DS` directives in the `DSEG` section. This distinction ensured correct memory segmentation and avoided naming conflicts. Multi-byte variables such as `int` were given two-byte space and accessed sequentially using indirect addressing through register `R0`. For example,

loading a global int variable involved initializing R0 to its address and reading its two bytes into working registers.

Register usage was planned to reflect the limitations of the 8051 architecture. Temporary values for intermediate expressions were stored in registers R1 through R7, starting from an offset index to avoid overlap with parameter or return value conventions. The system tracked the number of available registers to avoid overruns and ensured safe reuse. Arguments and return values followed the convention of using registers for function returns. This allowed for minimal memory access during function calls and improved efficiency. This strategy was verified through testing nested and multi-parameter function calls.

To support conditional and looping structures, the code generator dynamically created labels for branching. These were used in if, while, and for statements to mark the beginning and end of blocks, as well as jump targets for conditional expressions. Labels were uniquely numbered to avoid collisions across different parts of the code.

Testing confirmed that loops correctly returned to the appropriate condition-check label and exited as expected. Nested control flow also behaved correctly due to the stack-like nature of recursive code generation, maintaining context across subtrees.

Functions were declared and defined with proper scope and memory handling for parameters. Function calls pushed arguments into registers following a defined convention, and the return value was retrieved. Recursion was tested by simulating self-invoking functions and confirming that each frame correctly handled its own parameters and return path.

All generated assembly code matched the expected behavior defined in our initial design. The recursive traversal and structured handling of each node kind provided a modular and extensible way to support the full language feature set. Variable lifetime, scope resolution, register allocation, and branching logic were implemented consistently and passed all validation tests.

Chapter 6

Conclusion and Future work

The development of a C compiler targeting the ISA 8051 architecture marks a significant milestone in embedded systems programming for this classic microcontroller. This project has highlighted the complexity and meticulous effort required to adapt a high-level programming language like C to the unique constraints and features of the 8051 architecture, which is characterized by limited memory, registers, and instruction sets. Throughout this report, it became evident that efficiently translating C code into instructions compatible with the ISA 8051 demands a deep understanding of both the source language and the target architecture.

Furthermore, the compiler developed not only simplifies the process of embedded application development—making programming more accessible and productive for engineers—but also optimizes the use of the microcontroller’s limited resources, ensuring performance and energy efficiency. The implementation and testing phases demonstrated that successful integration between the C front-end and the ISA 8051 back-end is achievable, paving the way for future enhancements such as advanced optimizations, support for specialized libraries, and expansion to other microcontroller variants.

Looking ahead, several opportunities exist to extend and improve this compiler. Future work could focus on implementing more sophisticated optimization techniques tailored specifically for the 8051’s resource constraints, such as enhanced register allocation, loop unrolling, and instruction scheduling, which would further improve performance and reduce code size. Expanding language support to include a wider subset of C features and standard libraries would make the compiler suitable for more complex embedded applications. Integrating debugging and profiling tools would also be invaluable for developers aiming to optimize their code and troubleshoot effectively.

Additionally, broadening the compiler’s compatibility to other variants of the 8051 family could increase its practical applicability across a wider range of devices. Considering the growing prevalence of IoT and connected embedded systems, future iterations could incorporate code generation capabilities that facilitate integration with modern communication protocols or real-time operating systems, thereby

enhancing the compiler's relevance in contemporary embedded system development.

